



隙锋人形机器人

SDK 使用指南

目 录

前言	9
手册说明	9
读者对象	9
版权说明	9
商标声明	10
免责声明	10
修订记录	11
第1章 快速开始	14
1.1 使用前准备	14
1.1.1 运行环境要求	14
1.1.2 SDK 安装 & 导入	14
1.2 使用示例	14
1.2.1 AGV 控制	14
1.2.2 机械臂设置	17
第2章 接口说明	19
2.1 接口列表	19
2.2 初始化 & 连接管理	23
2.2.1 构造 SDK 实例	23
2.2.2 启动 SDK 服务 (start)	23
2.2.3 停止 SDK 服务 (stop)	24
2.2.4 获取连接状态 (getConnectionStatus)	24
2.2.5 请求连接机器人 (requestConnection)	25
2.2.6 关闭机器人 (shutDown)	26
2.2.7 重启机器人 (reboot)	26
2.2.8 复位机器人 (reset)	27
2.3 硬件控制	27
2.3.1 获取控制权信息 (getController)	27
2.3.2 切换操作模式 (switchOperationMode)	28
2.3.3 设置 AGV 速度数据 (setAgvVelData)	29
2.4 硬件信息 & 状态	29
2.4.1 获取电池信息 (getBatteryys)	29
2.4.2 获取按钮状态 (getButtonStatus)	30
2.4.3 获取 AGV 状态 (getAgvStatus)	31
2.4.4 获取 AGV 行驶状态 (getDriving)	32

2.4.5 获取操作模式 (getOperatingMode)	32
2.4.6 获取 AGV 软急停状态 (getSoftStopSwitch)	33
2.4.7 查询 AGV 基本信息 (queryAgvInfo)	34
2.4.8 查询数据库参数 (queryDatabaseParameter)	34
2.4.9 查询工业 PC 信息 (queryIndustrialPcInfo)	35
2.4.10 查询激光数据 (queryLaserData)	36
2.4.11 获取暂停状态 (getPaused)	36
2.4.12 获取复位状态 (getResettingStatus)	37
2.4.13 获取 RCS 连接状态 (getRcsConnectionState)	38
2.4.14 获取充电状态 (getChargeStatus)	38
2.4.15 获取驱动轮状态列表 (getDrivingWheels)	39
2.5 任务 & 动作	39
2.5.1 获取当前任务 ID (getTaskId)	39
2.5.2 获取任务更新 ID (getTaskUpdateId)	40
2.5.3 获取动作状态列表 (getActionStates)	41
2.5.4 获取最后到达的任务节点 ID (getLastNodeId)	41
2.5.5 获取最后到达的任务节点序列 ID (getLastNodeSequenceId)	42
2.5.6 获取最后进入的任务路径 ID (getLastEdgeId)	43
2.5.7 获取自上次任务节点以来的行驶距离 (getDistanceSinceLastNode)	43
2.5.8 触发已注册的任务链 (triggerTaskChain)	44
2.5.9 重新加载任务链定义文件 (reloadTaskChainFolder)	45
2.6 即时动作	46
2.6.1 重新定位 (reposition)	46
2.6.2 AGV 旋转 (vehicleRotation)	47
2.6.3 获取 AGV 位置 (getAgvPosition)	48
2.6.4 获取 AGV 速度 (getVelocity)	48
2.6.5 取消任务 (cancelTask)	49
2.6.6 暂停任务 (pauseTask)	49
2.6.7 恢复任务 (restoreTask)	50
2.6.8 启用指定地图(enableMap)	50
2.6.9 获取地图信息 (getMap)	51
2.6.10 下载地图 (downloadMap)	52
2.6.11 软急停控制 (softStop)	52
2.6.12 充电控制 (charge)	53
2.6.13 精准调整 (accurateAdjust)	54
2.6.14 对接操作 (docking)	55
2.6.15 清除错误 (clearError)	57

2.6.16 播放音频 (audioPlay)	58
2.6.17 读取车辆寄存器 (readVehicleRegister)	59
2.6.18 写入车辆寄存器 (writeVehicleRegister)	60
2.6.19 读取外部寄存器 (readExternalRegister)	60
2.6.20 写入外部寄存器 (writeExternalRegister)	61
2.6.21 校验外部寄存器 (checkExternalRegister)	62
2.6.22 切换避障场集 (switchAvoidanceFieldSet)	63
2.6.23 切换速度避障模式 (switchSpeedAvoidanceMode)	64
2.7 机械臂	64
2.7.1 机械臂关节坐标说明	65
2.7.2 初始化机械臂数据连接 (armInitData)	65
2.7.3 获取机械臂控制器连接状态 (armGetLinkStatus)	66
2.7.4 脱离示教器 (armDetachHMI)	66
2.7.5 获取脱离示教器状态 (armGetDetachHMIStatus)	67
2.7.6 获取伺服 OP 状态 (armServosOpOrNot)	67
2.7.7 获取报警信息 (armIsAlarming)	68
2.7.8 清除报警 (armClearAlarm)	68
2.7.9 机械臂使能控制 (armRobotEnableOrNot)	69
2.7.10 获取使能状态 (armGetRobotEnableStatus)	69
2.7.11 单个机械臂使能控制 (armSetSingleRobotEnable)	70
2.7.12 获取单个机械臂使能状态 (armGetSingleRobotEnableStatus)	71
2.7.13 设置全局速度 (armSetGlobalSpeed)	71
2.7.14 获取全局速度 (armGetGlobalSpeed)	72
2.7.15 设置报警时伺服是否下使能 (armSetRobotAlarmNotServoOff)	72
2.7.16 获取报警时伺服是否下使能 (armGetRobotAlarmNotServoOff)	73
2.7.17 设置报警提示配置 (armSetAlarmNotPrompt)	74
2.7.18 获取报警提示配置 (armGetAlarmNotPrompt)	74
2.7.19 腰部安全姿态约束 (armSetWaistSafePose)	75
2.7.20 获取腰部安全姿态约束 (armGetWaistSafePose)	75
2.7.21 设置机械臂保护状态 (armSetRobotProtectStatus)	76
2.7.22 获取机械臂保护状态 (armGetRobotProtectStatus)	77
2.7.23 打开串口 (armOpenCom)	78
2.7.24 读串口 (armReadCom)	79
2.7.25 写串口 (armWriteCom)	79
2.7.26 Modbus 控制 (armModbusControl)	80
2.7.27 机械臂直线运动 (armMoveWorlds)	83
2.7.28 机械臂弧线运动 (armMoveWorldsCurve)	84

2.7.29 机械臂整圆运动 (armMoveWorldsRound)	85
2.7.30 移动到指定关节坐标位置 (armMoveJoints)	86
2.7.31 角度转脉冲直接送伺服 (armPluseToServo)	87
2.7.32 世界坐标直接送伺服 (armWorldsToServo)	87
2.7.33 获取当前关节坐标位置 (armGetJoints)	88
2.7.34 获取当前世界坐标位置 (armGetWorlds)	89
2.7.35 获取当前规划速度 (armGetVelocity)	89
2.7.36 获取当前加速度 (armGetAcceleration)	90
2.7.37 切换运行模式 (armSwitchRunMode)	90
2.7.38 获取轴运行模式 (armGetCurrentRunMode)	91
2.7.39 获取反馈脉冲值 (armGetPulseFeedback)	92
2.7.40 获取机械臂力控扭矩 (armGetTorqueFeedbackVector)	92
2.7.41 获取轴力控扭矩 (armGetTorqueFeedbackSingle)	93
2.7.42 获取速度反馈 (armGetSpeedFeedback)	94
2.7.43 力矩模式控制 (armTorqueToServo)	94
2.7.44 速度模式控制 (armSpeedToServo)	95
2.7.45 暂停/恢复机械臂 (armPauseRobot)	95
2.7.46 清除机械臂路径规划 (armClearRobotRoute)	96
2.7.47 获取机械臂暂停状态 (armGetRobotPaused)	97
2.7.48 设置到位容差 (armSetEnableTolerance)	97
2.7.49 获取机械臂运动状态 (armGetRobotMoveState)	98
2.7.50 获取轴运动状态 (armGetRobotAxisMoveState)	99
2.7.51 获取轴极限值 (armGetAxisParameter)	99
2.7.52 设置轴极限值 (armSetAxisParameter)	100
2.7.53 世界坐标转关节坐标 (armTryWorlds)	101
2.7.54 检查关节是否超限 (armJointOutLimit)	101
2.7.55 开启/关闭拖动示教 (armActivateDragTeach)	102
2.7.56 开启/关闭碰撞检测 (armActivateCollision)	103
2.7.57 恒力模式切换 (armForceKeepSwitch)	103
2.7.58 获取力控状态 (armGetForceKeepState)	104
2.7.59 导纳控制初始化 (armAdmittanceControlInit)	105
2.7.60 导纳控制 (armAdmittanceControl)	106
2.7.61 获取当前绑定模式 (armGetArmBindMode)	106
2.7.62 绑定模式下世界坐标运动 (armBindModeWorldMove)	107
2.7.63 机械臂加载负载 (armSetRobotLoad)	109
2.7.64 机械臂卸载负载 (armUnsetLoad)	109
2.7.65 设置位置刷新周期 (armSetPosReFreshMS)	110

2.7.66 获取位置刷新周期 (armGetPosReFreshMS)	111
2.7.67 获取机械臂软急停状态 (armGetRobotSoftEmergencyStopStatus)	111
2.7.68 机械臂软急停控制 (armRobotSoftEmergencyStop)	112
2.8 系统管理 & 监控	113
2.8.1 RCS 连接管理 (rcsConnectionManage)	113
2.9 其他	114
2.9.1 获取扩展参数列表 (getExtendedParameters)	114
第3章 回调函数	115
3.1 回调函数类型定义	115
3.2 设置回调函数	115
3.2.1 设置 IMU 数据回调 (setImuCallback)	115
3.2.2 设置里程计数据回调 (setOdomCallback)	116
3.2.3 设置速度数据回调 (setVelocityCallback)	117
3.2.4 设置当前位姿回调 (setCurrentPoseCallback)	117
3.2.5 设置二维码数据回调 (setQrDataCallback)	118
3.2.6 设置相机数据回调 (setCameraCallback)	119
3.2.7 设置激光雷达数据回调 (setLidarDataCallback)	120
3.2.8 设置关节运动结束回调 (armSetMoveJointsEndCallback)	120
3.2.9 设置世界运动结束回调 (armSetMoveWorldsEndCallback)	121
3.3 取消回调函数	122
3.3.1 通用取消回调 (cancelCallback)	122
3.3.2 取消 IMU 回调 (cancelImuCallback)	123
3.3.3 取消里程计回调 (cancelOdomCallback)	124
3.3.4 取消速度回调 (cancelVelocityCallback)	124
3.3.5 取消当前位姿回调 (cancelCurrentPoseCallback)	125
3.3.6 取消二维码数据回调 (cancelQrDataCallback)	125
3.3.7 取消相机数据回调 (cancelCameraCallback)	126
3.3.8 取消激光雷达数据回调 (cancelLidarDataCallback)	126
3.3.9 取消关节运动结束回调 (armCancelMoveJointsEndCallback)	127
3.3.10 取消世界运动结束回调 (armCancelMoveWorldsEndCallback)	127
第4章 数据类型	129
4.1 数据类型列表	129
4.2 基础数据	130
4.2.1 位置信息 (Position)	130
4.2.2 速度信息 (Velocity)	131
4.2.3 地图信息 (MapInfo)	131
4.3 硬件状态数据	132

4.3.1 动作状态 (ActionState)	132
4.3.2 电池信息 (BatteryInfo)	133
4.3.3 错误信息 (ErrorInfo)	135
4.3.4 按钮状态 (ButtonStatus)	135
4.3.5 扩展参数 (Parameter)	136
4.3.6 驱动轮状态 (DrivingWheelInfo)	136
4.4 查询和系统信息数据	137
4.4.1 二维码信息 (QrInfo)	137
4.4.2 AGV 基本信息 (AgvInfo)	137
4.4.3 计算机信息 (ComputerInfo)	138
4.5 几何数据	139
4.5.1 二维点 (Point2D)	139
4.5.2 三维点 (Point3D)	139
4.5.3 二维激光数据 (LaserData2D)	140
4.6 传感器数据	140
4.6.1 IMU 数据 (ImuData)	140
4.6.2 里程计数据 (OdomData)	141
4.6.3 激光雷达数据 (LidarData)	141
4.6.4 速度数据 (VelocityData)	142
4.6.5 当前位姿数据 (CurrentPoseData)	143
4.6.6 二维码数据 (QrData)	143
4.6.7 相机数据 (CameraData)	144
4.7 机械臂相关数据	145
4.7.1 轴参数索引 (AxisParameterIndex)	145
4.7.2 手臂绑定模式 (ArmBindMode)	145
4.7.3 负载信息 (LoadInfo)	146
4.7.4 机械臂世界坐标数据 (RobotWorldsData)	146
4.7.5 轴运动结束回调数据 (MoveJointsEndData)	146
4.7.6 世界坐标运动结束回调数据 (MoveWorldsEndData)	147
4.8 VECTOR 类型说明	147
4.8.1 浮点数向量 (FloatVector)	147
4.8.2 双精度浮点数向量 (DoubleVector)	148
4.8.3 整数向量 (IntVector)	148
4.8.4 字符串向量 (StringVector)	148
4.8.5 布尔向量 (BoolVector)	149
4.8.6 字符串映射 (StringMap)	149
第5章 错误处理	150

5.1 错误码说明 150

5.2 常见错误处理模式 150

5.3 异常处理 151

5.4 连接状态检查 151

第6章 联系我们..... 153

前言

手册说明

该 SDK 是一套用于与“隙锋”人形机器人进行程序化交互的 Python SDK。通过该 SDK，你可以在无需深入底层控制系统的前提下，完成以下典型能力：

- **连接机器人：**建立与机器人本体或控制系统的通信。
- **获取实时状态：**位置、速度、电量、地图、任务状态。
- **监控任务执行：**动作状态、路径节点、运行模式。
- **接入上位系统：**MES / WMS / 调度系统 / 可视化监控平台。
- **二次开发与验证：**仿真联调、算法验证、自动化测试。

读者对象

本手册适用于：

- 上位系统 / 平台软件开发
- 系统集成商（机器人调度、产线对接）
- 算法工程师（感知 / 决策 / 评估）
- 算法工程师（感知 / 决策 / 评估）

如果你会 Python，且希望以**最小成本对接机器人能力**，本 SDK 即为推荐入口。

版权说明

深圳优艾智合机器人科技有限公司具备本产品及其软件的专利权、版权及其它知识产权，未经本公司书面授权，任何单位和个人不得擅自摘抄、复制本文档内容；不得直接或间接复制、制造、加工、使用本产品及其相关部分。

本手册以及相关内容仅用于为最终客户提供产品使用信息，深圳优艾智合机器人科技有限公司有权随时更改或撤销其内容。手册是深圳优艾智合机器人科技有限公司的专有信息，并受中华人民共和国版权法和国际公约的保护。

商标声明

本手册中提及的所有商标、商标名称、服务标志、产品外观及徽标均归其各自公司所有。

有关法律允许的范围内，深圳优艾智合机器人科技有限公司按照“既定”方式提供本手册，且不对本手册提供任何形式的担保，包括（但不限于）对特定目的的适销性、适用性以及无侵权行为不作任何暗示担保。在任何情况下，深圳优艾智合机器人科技有限公司不对最终客户或任何第三方因使用本手册而引发的任何直接或者间接的损失负责。包括（但不限于）利润损失、业务中断、商誉影响或财产损失等，即使已将此类损失或损坏明确告知深圳优艾智合机器人科技有限公司。

免责声明

本手册内容若有变动，恕不另行通知。本手册例子中使用公司、人名数据若非特别声明，均为虚构。

本手册以及本手册所提的任何产品的使用均受使用的用户许可协议限制。

本产品的使用和维修具有很强专业性，未经深圳优艾智合机器人科技有限公司培训及授权的单位及个人不得擅自维修和拆解本产品。本文档不具备维修检测指导作用，仅作为使用说明文档。

在适用法律允许的最大范围内，本手册及其内容均按“现状”提供，本公司明示不对其作出任何形式的担保（无论是明示、默示或法定的），包括但不限于对适销性、适用于某一特定用途的适用性以及不侵犯第三方权利的默示担保。

无论在任何情况下，本公司及关联方均不对因使用或无法使用本手册、本产品而引发的任何直接、间接、附带、特殊、后果性损失（包括但不限于利润损失、业务中断、商誉损害、数据丢失或其他财产损失）承担任何责任，即使本公司已被告

知发生此类损失的可能性。

运行中的设备具备危险！使用者有责任在设备运行过程中制定有效的安全保护机制，深圳优艾智合机器人科技有限公司没有义务和责任对此造成的附带或相应产生的损失负责。

深圳优艾智合机器人科技有限公司不承担由于错误使用本产品,所造成的直接、间接、特殊、附带等产生的损失和责任。

修订记录

版本	日期	描述
V1.0	2026/02/04	首次发布
V260422	2026/05/18	配套 2026.04.22 发布的 SDK
V260422.3	2026/05/19	armMoveWorlds 增加腰部控制注意事项
V260422.4	2026/05/19	更新使用示例
V260422.5	2026/05/20	修改机械臂关节坐标相关说明
V260526.1	2026/06/04	配套 2.0.0.0526 版本的 SDK： 1. Python 版本支持变更为 3.8 ~ 3.12 2. 接口变更 新增 ● 获取暂停状态（getPaused） ● 获取复位状态（getResettingStatus） ● 获取 RCS 连接状态（getRcsConnectionState） ● 获取充电状态（getChargeStatus） ● 获取驱动轮状态列表（getDrivingWheels） ● 清除错误（clearError） ● 获取机械臂力控扭矩（armGetTorqueFeedbackVector）

版本	日期	描述
		● 获取轴力控扭矩 (armGetTorqueFeedbackSingle) ● 机械臂加载负载 (armSetRobotLoad) ● 机械臂卸载负载 (armUnsetLoad) ● 设置位置刷新周期 (armSetPosReFreshMS) ● 获取位置刷新周期 (armGetPosReFreshMS) ● 获取机械臂软急停状态 (armGetRobotSoftEmergencyStopStatus) ● 机械臂软急停控制 (armRobotSoftEmergencyStop) 修改 ● 切换操作模式 (switchOperationMode) ● 获取操作模式 (getOperatingMode) ● 获取 AGV 状态 (getAgvStatus) ● 软急停控制 (softStop) ● 充电控制 (charge) ● 对接操作 (docking) ● 获取机械臂控制器连接状态 (armGetLinkStatus) ● 设置报警时伺服是否下使能 (armSetRobotAlarmNotServoOff) ● 获取机械臂保护状态 (armGetRobotProtectStatus) ● 开启/关闭碰撞检测 (armActivateCollision) 删除 ● 触发建图 (buildingMap) ● 清除对接错误 (clearDockingError) ● 获取手动控制动作列表 (getManualControlActions) ● 监控手动释放 (monitoringManualRelease)

版本	日期	描述
		4. 数据类型变更 新增 ● 驱动轮状态 (DrivingWheelInfo) 修改 ● 动作状态 (ActionState) ● 电池信息 (BatteryInfo) 删除 ● 手动控制动作状态 (ManualControlAction) 4. 世界坐标和关节角度的参数类型由 list 类型改为 Vector 类型

第1章 快速开始

1.1 使用前准备

1.1.1 运行环境要求

- Python 版本支持: 3.8 ~ 3.12
- 双架构支持: x86_64, aarch64

1.1.2 SDK 安装 & 导入

```
# 安装, 包名请根据实际修改
pip install pypilot-2.0.0.0526-cp38-cp38-linux_x86_64.whl

# 导入
import pypilot
from pypilot import PilotSDK, Position, Velocity, MapInfo
```

1.2 使用示例

1.2.1 AGV 控制

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-

import pypilot
import time
import math

def main():
```

```
# 创建 SDK 实例
sdk = pypilot.PilotSDK("192.168.1.100")

try:
    # 启动 SDK
    if not sdk.start():
        print("SDK 启动失败")
        return

    print("SDK 启动成功")

    result = sdk.requestConnection("192.168.0.100")
    if result != 0:
        print("请求连接失败")
        return
    print(f"Connection result: {result}")

    # 设置回调函数
    def on_imu_data(imu_data):
        print(f"IMU: 角速度({imu_data.angular_vel_x:.2f}, {imu_data.angular_vel_y:.2f}, {imu_data.angular_vel_z:.2f})")

    def on_odom_data(odom_data):
        print(f"位置: ({odom_data.pos_x:.2f}, {odom_data.pos_y:.2f})")

    sdk.setImuCallback(on_imu_data)
    sdk.setOdomCallback(on_odom_data)

    # 获取初始状态
    result_code, position = sdk.getAgvPosition()
    if result_code == 0:
        print(f"初始位置: ({position.x:.2f}, {position.y:.2f}, {math.degrees(position.theta):.2f}°)")

    result_code, velocity = sdk.getVelocity()
    if result_code == 0:
        print(f"当前速度: ({velocity.vx:.2f}, {velocity.vy:.2f}, {velocity.omega:.2f})")
```

```
# 旋转90度
print("旋转 90 度...")
sdk.vehicleRotation(math.pi/2, "MAP")
time.sleep(3)

# 获取最终位置
result_code, position = sdk.getAgvPosition()
if result_code == 0:
    print(f"最终位置: ({position.x:.2f}, {position.y:.2f}, {math.degrees(position.theta):.2f}°)")

# 获取电池状态
result_code, batteries = sdk.getBatterys()
if result_code == 0:
    for battery in batteries:
        print(f"电池电量: {battery.batteryLevel}%")
        print(f"充电状态: {'充电中' if battery.charging else '未充电'}")

except KeyboardInterrupt:
    print("用户中断")
except Exception as e:
    print(f"发生错误: {e}")
finally:
    # 清理资源
    print("停止运动...")
    sdk.manualSpeedControl("DEFAULT", 0.0, 0.0, 0.0)

    print("取消回调...")
    sdk.cancelImuCallback()
    sdk.cancelOdomCallback()

    print("停止 SDK...")
    sdk.stop()

    print("程序结束")

if __name__ == "__main__":
```

```
main()
```

1.2.2 机械臂设置

```
sdk = pypilot.PilotSDK("192.168.1.100")
# 初始化机械臂数据连接
sdk.armInitData("192.168.1.100", "192.168.1.200", 8080)

# 脱离示教器
sdk.armDetachHMI()

# 清除报警
while sdk.armIsAlarming()[0] == 0:
    logger.info("当前有报警清除报警\n")
    sdk.armClearAlarm()
    time.sleep(1)

# 机器人使能
sdk.armRobotEnableOrNot(True)

# 4. 设置全局速度
sdk.armSetGlobalSpeed(10.0)

# 5. 移动到指定位置 (世界坐标 XYZUVW)
result, current_worlds = sdk.armGetWorlds(1)
if result != 0:
    print(f"或其世界坐标系失败")
    exit(0)

print(f"current_worlds: {current_worlds[0]} {current_worlds[1]}
{current_worlds[2]} {current_worlds[3]} {current_worlds[4]}
{current_worlds[5]}")

# 机器人 1 (左臂) 向 Z 正方向移动 20mm
current_worlds[2] += 20
```

```
# 执行机器人1 移动
sdk.armMoveWorlds(1, current_worlds)
target_z = current_worlds[2]

# 等待执行完成
while True:
    time.sleep(0.1)
    current_pose = sdk.getRobotWorld(1)
    if current_pose:
        if math.fabs(current_pose[2] - target_z) < 3:
            break

# 打开控制器COM1 口
sdk.armOpenCom("COM1", 115200, 8, 0, 1)

# 夹爪1 闭合
data = bytes.fromhex("eb 90 01 05 10 f4 01 64 00 6f")
sdk.armWriteCom("COM1", data)
# 夹爪2 闭合
data = bytes.fromhex("eb 90 02 05 10 f4 01 64 00 70")
sdk.armWriteCom("COM1", data)
time.sleep(5)

# 夹爪1 打开
data = bytes.fromhex("eb 90 01 03 11 f4 01 0a")
sdk.armWriteCom("COM1", data)
# 夹爪2 打开
data = bytes.fromhex("eb 90 02 03 11 f4 01 0b")
sdk.armWriteCom("COM1", data)

# 下使能
sdk.armRobotEnableOrNot(False)
```

第2章 接口说明

2.1 接口列表

模块	接口
初始化 & 连接管理	● 启动/停止 SDK 服务 ● 获取连接状态 ● 请求连接/关闭/重启/复位机器人
硬件控制	● 获取控制权信息 ● 切换操作模式 ● 设置 AGV 速度数据
硬件信息 & 状态	● 获取电池信息 ● 获取按钮状态 ● 获取 AGV 状态/行驶状态 ● 获取操作模式 ● 获取 AGV 软急停状态 ● 查询 AGV 基本信息 ● 查询数据库参数 ● 查询工业 PC 信息 ● 查询激光数据 ● 获取暂停状态 ● 获取复位状态 ● 获取 RCS 连接状态 ● 获取充电状态 ● 获取驱动轮状态列表
任务 & 动作	● 获取任务 ID/任务更新 ID ● 获取动作状态列表 ● 获取最后到达的任务节点 ID/任务节点序列 ID ● 获取最后进入的任务路径 ID

模块	接口
	● 获取自上次任务节点以来的行驶距离 ● 触发已注册的任务链 ● 重新加载任务链定义文件
即时动作	● 重新定位 ● AGV 旋转 ● 获取 AGV 位置/速度 ● 取消/暂停/恢复任务 ● 启用指定地图 ● 获取地图信息 ● 下载地图 ● 触发建图 ● 软急停控制 ● 充电控制 ● 精准调整 ● 对接操作 ● 清除错误 ● 播放音频 ● 读取/写入车辆寄存器 ● 读取/写入/校验外部寄存器 ● 切换避障场集/速度避障模式
机械臂	基础函数 - 初始化与状态管理 ● 初始化机械臂数据连接 ● 获取下位机连接状态 ● 脱离示教器 ● 获取脱离示教器状态 ● 获取伺服 OP 状态 ● 获取报警信息 ● 清除报警 ● 机器人使能控制 ● 获取使能状态

模块	接口
	● 单个机器人使能控制 ● 获取单个机器人使能状态 ● 设置全局速度(0-100) ● 获取全局速度 ● 报警时伺服是否下使能 ● 获取报警时伺服是否下使能 ● 设置报警提示配置 ● 获取报警提示配置 ● 腰部安全姿态约束 ● 获取腰部安全姿态约束 ● 设置机器人保护状态 ● 获取机器人保护状态 ● 打开/读/写串口 ● Modbus 控制 运动函数 - 运动控制 ● 机械臂直线运动 ● 机械臂弧线运动 ● 机械臂整圆运动 ● 移动到指定关节坐标位置 ● 角度转脉冲直接送伺服 ● 末端坐标直接送伺服 ● 获取当前关节坐标位置 ● 获取当前世界坐标位置 ● 获取当前规划速度 ● 获取当前加速度 ● 切换运行模式 ● 获取轴运行模式 ● 获取反馈脉冲值 ● 获取机械臂力控扭矩 ● 获取轴力控扭矩

模块	接口
	● 获取速度反馈 ● 力矩模式控制 ● 速度模式控制 ● 暂停/恢复机器人 ● 清除机器人路径 ● 判断机器人是否暂停 ● 设置到位容差 ● 获取机器人运动状态 ● 获取轴运动状态 ● 获取轴极限值 ● 设置轴极限值 ● 末端坐标转关节坐标 ● 检查关节是否超限 力控函数 - 力控与碰撞检测 ● 开启/关闭拖动示教 ● 开启/关闭碰撞检测 ● 恒力模式切换 ● 获取力控状态 ● 导纳控制初始化 ● 导纳控制 自定义函数 - 绑定模式与腰部控制 ● 获取当前绑定模式 ● 绑定模式下末端坐标运动 其他函数 ● 机械臂加载负载 ● 机械臂卸载负载 ● 设置位置刷新周期 ● 获取位置刷新周期 ● 获取机械臂软急停状态 ● 机械臂软急停控制

模块	接口
系统管理 & 监控	● RCS 连接管理
其他	● 获取扩展参数列表

2.2 初始化 & 连接管理

2.2.1 构造 SDK 实例

```
def __init__(self, robotHost: str = "127.0.0.1") -> None:
```

参数说明:

- **robotHost: str** - 机器人 IP 地址, 默认为本地地址"127.0.0.1"

使用示例:

```
# 连接本地机器人
sdk = PilotSDK()

# 连接指定 IP 的机器人
sdk = PilotSDK("192.168.1.100")
```

2.2.2 启动 SDK 服务 (start)

```
def start(self, wsServerThreads: int = 1) -> int:
```

参数说明:

- **wsServerThreads: int** - WebSocket 服务器线程数, 默认为 1

返回值:

- **int** - 启动成功返回 0, 失败返回非 0

使用示例:

```
sdk = PilotSDK("192.168.1.100")
if sdk.start() == 0:
    print("SDK 启动成功")
else:
    print("SDK 启动失败")
```

2.2.3 停止 SDK 服务 (stop)

```
def stop(self) -> None:
```

使用示例:

```
sdk.stop()
print("SDK 已停止")
```

2.2.4 获取连接状态 (getConnectionStatus)

```
def getConnectionStatus(self) -> str:
```

返回值:

- **str** - JSON 格式的连接状态信息

使用示例:

```
status = sdk.getConnectionStatus()
print(f"连接状态: {status}")
```

2.2.5 请求连接机器人 (requestConnection)

```
def requestConnection(self, ip: str, port: str = None) -> int:
```

参数说明:

- **ip: str** - 本地电脑 IP 地址
- **port: str** - 机器人端口 (可选, 不填写时会使用[8000,9000]范围内的随机端口)

返回值:

- **int** - 结果码, 0 表示成功, 非 0 表示失败

使用示例:

```
# 使用默认端口连接
result = sdk.requestConnection("192.168.1.100")

# 指定端口连接
result = sdk.requestConnection("192.168.1.100", "8858")
if result == 0:
    print("连接请求成功")
```

2.2.6 关闭机器人 (shutDown)

```
def shutDown(self) -> int:
```

返回值:

- **int** - 结果码, 0 表示成功, 非 0 表示失败

使用示例:

```
result = sdk.shutDown()
if result == 0:
    print("机器人关闭命令发送成功")
```

2.2.7 重启机器人 (reboot)

```
def reboot(self) -> int:
```

返回值:

- **int** - 结果码, 0 表示成功, 非 0 表示失败

使用示例:

```
result = sdk.reboot()
if result == 0:
    print("机器人重启命令发送成功")
```

2.2.8 复位机器人 (reset)

```
def reset(self) -> int:
```

返回值:

- **int** - 结果码, 0 表示成功, 非 0 表示失败

使用示例:

```
result = sdk.reset()
if result == 0:
    print("机器人复位命令发送成功")
```

2.3 硬件控制

2.3.1 获取控制权信息 (getController)

```
def getController(self) -> tuple[int, str]:
```

返回值:

- **tuple[int, str]** - (结果码, 控制器信息)
 - 结果码为 0 表示成功, 非 0 表示失败
 - 控制器信息为字符串, 表示当前控制权的归属标识符

使用示例:

```
result_code, controller = sdk.getController()
```

```
if result_code == 0:
    print(f"当前控制权归属: {controller}")
    if controller == "NULL":
        print("无人控制")
    else:
        print(f"控制者: {controller}")
```

2.3.2 切换操作模式 (switchOperationMode)

```
def switchOperationMode(self, mode: str) -> int:
```

参数说明:

- **mode: str** - 操作模式
 - AUTO: 自动模式
 - MANUAL: 手动模式

返回值:

- **int** - 结果码, 0 表示成功, 非 0 表示失败

使用示例:

```
# 切换到自动模式
result = sdk.switchOperationMode("AUTO")
if result == 0:
    print("已切换到自动模式")

# 切换到手动模式
result = sdk.switchOperationMode("MANUAL")
if result == 0:
    print("已切换到手动模式")
```

2.3.3 设置 AGV 速度数据 (setAgvVelData)

```
def setAgvVelData(self, x: float, y: float, omega: float) -> int:
```

参数说明:

- **x: float** - X 方向速度 (米/秒)
- **y: float** - Y 方向速度 (米/秒)
- **omega: float** - 角速度 (弧度/秒)

返回值:

- **int** - 结果码, 0 表示成功, 非 0 表示失败

使用示例:

```
result = sdk.setAgvVelData(1.0, 0.0, 0.5) # 前进1m/s, 旋转0.5rad/s
if result == 0:
    print("速度数据设置成功")
```

2.4 硬件信息 & 状态

2.4.1 获取电池信息 (getBatteryInfo)

```
def getBatteryInfo(self) -> tuple[int, List[BatteryInfo]]:
```

返回值:

- **tuple[int, List[BatteryInfo]]** - (结果码, 电池信息列表)
 - 结果码为 0 表示成功, 非 0 表示失败
 - 电池信息列表包含的信息详见第 4 章 数据类型

使用示例:

```
result_code, batteries = sdk.getBatterys()
if result_code == 0:
    for i, battery in enumerate(batteries):
        print(f"电池{i+1}: ID={battery.batteryId}")
        print(f" 电量: {battery.batteryLevel}%")
        print(f" 电压: {battery.batteryVoltage}V")
        print(f" 健康度: {battery.batteryHealth}/100")
        print(f" 充电状态: {'充电中' if battery.charging else '未充电'})
```

2.4.2 获取按钮状态 (getButtonStatus)

```
def getButtonStatus(self) -> tuple[int, ButtonStatus]:
```

返回值:

- **tuple[int, ButtonStatus]** - (结果码, 按钮状态对象)
 - 结果码为 0 表示成功, 非 0 表示失败
 - 按钮状态对象详见第 4 章 数据类型

使用示例:

```
result_code, button_status = sdk.getButtonStatus()
if result_code == 0:
    print(f"电源按钮状态: {button_status.powerButton}")
```

```
print(f"复位按钮状态: {button_status.resetButton}")
```

2.4.3 获取 AGV 状态 (getAgvStatus)

```
def getAgvStatus(self) -> tuple[int, agvStatus]:
```

返回值:

- **tuple[int, agvStatus]** - (结果码, AGV 状态)
 - 结果码为 0 表示成功, 非 0 表示失败
 - AGV 状态: IDLE (空闲) /RUNNING (正在运行) /OFFLINE (离线) /ERROR (报错)

使用示例:

```
result_code, agv_status = sdk.getAgvStatus()
if result_code == 0:
    # 获取 AGV 状态
    print(f"AGV 状态: {agv_status}")
    if agv_status == "IDLE":
        print("AGV 空闲")
    elif agv_status == "RUNNING":
        print("AGV 正在运行")
    elif agv_status == "OFFLINE":
        print("AGV 离线")
else:
    print(f"获取 AGV 状态失败, 错误码: {result_code}")
```

2.4.4 获取 AGV 行驶状态 (getDriving)

```
def getDriving(self) -> tuple[int, bool]:
```

返回值:

- **tuple[int, bool]** - (结果码, 是否正在行驶)
 - 结果码为 0 表示成功, 非 0 表示失败
 - 是否正在行驶为 True 表示在行驶, False 表示静止状态

使用示例:

```
result_code, is_driving = sdk.getDriving()
if result_code == 0:
    if is_driving:
        print("AGV 正在行驶")
    else:
        print("AGV 处于静止状态")
```

2.4.5 获取操作模式 (getOperatingMode)

```
def getOperatingMode(self) -> tuple[int, str]:
```

返回值:

- **tuple[int, str]** - (结果码, 操作模式)
 - 结果码为 0 表示成功, 非 0 表示失败
 - 操作模式: AUTO (自动模式) / MANUAL (手动模式) / MAINTENANCE (维

修模式)

使用示例:

```
result_code, mode = sdk.getOperatingMode()
if result_code == 0:
    print(f"操作模式: {mode}")
    if mode == "AUTO":
        print("当前为自动模式")
    elif mode == "MANUAL":
        print("当前为手动模式")
    elif mode == "MAINTENANCE":
        print("当前为维修模式")
```

2.4.6 获取 AGV 软急停状态 (getSoftStopSwitch)

```
def getSoftStopSwitch(self) -> tuple[int, str]:
```

返回值:

- **tuple[int, str]** - (结果码, 软急停开关状态)
 - 结果码为 0 表示成功, 非 0 表示失败
 - 软急停开关状态: CLOSE (未触发急停) / OPEN (急停中)

使用示例:

```
result_code, soft_stop = sdk.getSoftStopSwitch()
if result_code == 0:
    print(f"软急停状态: {soft_stop}")
    if soft_stop == "OPEN":
        print("软急停已触发, AGV 无法运动")
    else:
```

```
print("软急停未触发")
```

2.4.7 查询 AGV 基本信息 (queryAgvInfo)

```
def queryAgvInfo(self) -> tuple[int, AgvInfo]:
```

返回值:

- **tuple[int, AgvInfo]** - (结果码, AGV 信息对象)
 - 结果码为 0 表示成功, 非 0 表示失败
 - AGV 信息对象详见第 4 章 数据类型

使用示例:

```
result_code, agv_info = sdk.queryAgvInfo()
if result_code == 0:
    print(f"AGV ID: {agv_info.agvId}")
    print(f"系列名称: {agv_info.seriesName}")
    print(f"序列号: {agv_info.serialNumber}")
    for key, value in agv_info.versions.items():
        print(f"版本 {key}: {value}")
```

2.4.8 查询数据库参数 (queryDatabaseParameter)

```
def queryDatabaseParameter(self, paramName: str) -> tuple[int, str]:
```

参数说明:

- **paramName: str** – 要查询的参数名称，具体列表请联系技术支持获取。

返回值:

- **tuple[int, str]** – (结果码, 参数值)
 - 结果码为 0 表示成功, 非 0 表示失败
 - 查询的参数在数值库中的值

使用示例:

```
result_code, param_value = sdk.queryDatabaseParameter("max_speed")
if result_code == 0:
    print(f"参数 max_speed 的值: {param_value}")
```

2.4.9 查询工业 PC 信息 (queryIndustrialPcInfo)

```
def queryIndustrialPcInfo(self) -> tuple[int, ComputerInfo]:
```

返回值:

- **tuple[int, ComputerInfo]** – (结果码, 计算机信息对象)
 - 结果码为 0 表示成功, 非 0 表示失败
 - 计算机信息对象详见第 4 章 数据类型

使用示例:

```
result_code, pc_info = sdk.queryIndustrialPcInfo()
if result_code == 0:
    print(f"CPU 占用率: {pc_info.cpu_ocp}%")
    print(f"内存占用率: {pc_info.mem_ocp}%")
    print(f"剩余磁盘空间: {pc_info.space_free}GB")
    print(f"总磁盘空间: {pc_info.space_total}GB")
```

```
print(f"温度: {pc_info.temperature}°C")
print(f"CPU 信息: {pc_info.CPU}")
print(f"系统信息: {pc_info.system}")
print(f"硬盘寿命: {pc_info.harddisk_life}")
```

2.4.10 查询激光数据 (queryLaserData)

```
def queryLaserData(self) -> tuple[int, LaserData2D]:
```

返回值:

- **tuple[int, LaserData2D]** - (结果码, 激光数据对象)
 - 结果码为 0 表示成功, 非 0 表示失败
 - 激光数据对象详见第 4 章 数据类型

使用示例:

```
result_code, laser_data = sdk.queryLaserData()
if result_code == 0:
    print(f"激光点数量: {len(laser_data.points)}")
    for i, point in enumerate(laser_data.points[:5]): # 显示前 5 个点
        print(f"点{i+1}: ({point.x:.2f}, {point.y:.2f})")
```

2.4.11 获取暂停状态 (getPaused)

```
def getPaused() -> tuple[int, bool]
```

返回值:

- **tuple[int, bool]** - (结果码, 暂停状态)
 - 结果码为 0 表示成功, 非 0 表示失败
 - 暂停状态为 True 表示暂停中

使用示例:

```
result_code, status = sdk.queryLaserData()
if result_code == 0 and status:
    print(f"机器人处于暂停状态")
```

2.4.12 获取复位状态 (getResettingStatus)

```
def getResettingStatus() -> tuple[int, str]
```

返回值:

- **tuple[int, str]** - (结果码, 复位状态)
 - 结果码为 0 表示成功, 非 0 表示失败
 - 复位状态: NORMAL (正常) /PRESSED (按钮已按下) /RESETTING (复位中) /AWAITING_RESET (待复位)

使用示例:

```
result_code, status = sdk.getResettingStatus()
if result_code == 0 and status == "AWAITING_RESET":
    print(f"机器人处于待复位状态")
```

2.4.13 获取 RCS 连接状态 (getRcsConnectionState)

```
def getRcsConnectionState() -> tuple[int, str]
```

返回值:

- **tuple[int, str]** - (结果码, RCS 连接状态)
 - 结果码为 0 表示成功, 非 0 表示失败
 - RCS 连接状态: OFFLINE (离线) / ONLINE (在线) / CONNECTING (连接中)

使用示例:

```
result_code, status = sdk.getRcsConnectionState()
if result_code == 0 and status == "ONLINE":
    print(f"RCS 在线")
```

2.4.14 获取充电状态 (getChargeStatus)

```
def getChargeStatus() -> tuple[int, bool]
```

返回值:

- **tuple[int, bool]** - (结果码, 充电状态)
 - 结果码为 0 表示成功, 非 0 表示失败
 - 充电状态为 True 表示充电中

使用示例:

```
result_code, status = sdk.getChargeStatus()
if result_code == 0 and status:
    print(f"机器人处于充电状态")
```

2.4.15 获取驱动轮状态列表 (getDrivingWheels)

```
def getDrivingWheels() -> tuple[int, List[DrivingWheelInfo]]
```

返回值:

- **tuple[int, List[DrivingWheelInfo]]** - (结果码, 驱动轮状态列表)
 - 结果码为 0 表示成功, 非 0 表示失败
 - 驱动轮状态对象详见第 4 章 数据类型

使用示例:

```
result_code, status = sdk.getDrivingWheels()
if result_code == 0 and len(status) > 0:
    print(f"机器人驱动轮{status[0].drivingWheelId}的状态为{status[0].status}")
```

2.5 任务 & 动作

2.5.1 获取当前任务 ID (getTaskId)

```
def getTaskId(self) -> tuple[int, str]:
```

返回值:

- **tuple[int, str]** - (结果码, 任务 ID)
 - 结果码为 0 表示成功, 非 0 表示失败
 - 任务 ID 为字符串形式的任务唯一标识符

使用示例:

```
result_code, task_id = sdk.getTaskId()
if result_code == 0:
    print(f"当前任务 ID: {task_id}")
else:
    print(f"获取任务 ID 失败, 错误码: {result_code}")
```

2.5.2 获取任务更新 ID (getTaskUpdateId)

```
def getTaskUpdateId(self) -> tuple[int, int]:
```

返回值:

- **tuple[int, int]** - (结果码, 任务更新 ID)
 - 结果码为 0 表示成功, 非 0 表示失败
 - 任务更新 ID 用于标识任务的更新次数, 初始为 0, 每次更新加 1

使用示例:

```
result_code, update_id = sdk.getTaskUpdateId()
if result_code == 0:
    print(f"任务更新 ID: {update_id}")
```

2.5.3 获取动作状态列表 (getActionStates)

```
def getActionStates(self) -> tuple[int, List[ActionState]]:
```

返回值:

- **tuple[int, List[ActionState]]** - (结果码, 动作状态列表)
 - 结果码为 0 表示成功, 非 0 表示失败
 - 动作状态列表包含最近的 200 个动作状态, 数据类型详见第 4 章 数据类型

使用示例:

```
result_code, action_states = sdk.getActionStates()
if result_code == 0:
    print(f"动作状态数量: {len(action_states)}")
    for i, action in enumerate(action_states[-5:]): # 显示最近 5 个动作
        print(f"动作{i+1}: ID={action.actionId}, 类型={action.actionType}")
        print(f"  状态: {action.actionStatus}")
        print(f"  描述: {action.actionDescription}")
```

2.5.4 获取最后到达的任务节点 ID (getLastNodeId)

```
def getLastNodeId(self) -> tuple[int, str]:
```

返回值:

- **tuple[int, str]** - (结果码, 节点 ID)

- 结果码为 0 表示成功，非 0 表示失败
- 节点 ID 为字符串形式的路径节点唯一标识符

使用示例:

```
result_code, node_id = sdk.getLastNodeId()
if result_code == 0:
    print(f"最后到达的节点: {node_id}")
else:
    print(f"获取节点 ID 失败, 错误码: {result_code}")
```

2.5.5 获取最后到达的任务节点序列 ID (getLastNodeSequenceId)

```
def getLastNodeSequenceId(self) -> tuple[int, int]:
```

返回值:

- **tuple[int, int]** - (结果码, 节点序列 ID)
 - 结果码为 0 表示成功，非 0 表示失败
 - 节点序列 ID 为整数，用于跟踪订单中节点的执行顺序

使用示例:

```
result_code, sequence_id = sdk.getLastNodeSequenceId()
if result_code == 0:
    print(f"最后节点的序列 ID: {sequence_id}")
```

2.5.6 获取最后进入的任务路径 ID (getLastEdgeId)

```
def getLastEdgeId(self) -> tuple[int, str]:
```

返回值:

- **tuple[int, str]** - (结果码, 路径 ID)
 - 结果码为 0 表示成功, 非 0 表示失败
 - 路径 ID 为字符串形式的路径段唯一标识符

使用示例:

```
result_code, edge_id = sdk.getLastEdgeId()
if result_code == 0:
    print(f"最后进入的路径: {edge_id}")
```

2.5.7 获取自上次任务节点以来的行驶距离 (getDistanceSinceLastNode)

```
def getDistanceSinceLastNode(self) -> tuple[int, float]:
```

返回值:

- **tuple[int, float]** - (结果码, 行驶距离)
 - 结果码为 0 表示成功, 非 0 表示失败
 - 行驶距离为浮点数, 单位为米, 表示车辆超过最后节点的距离

使用示例:

```
result_code, distance = sdk.getDistanceSinceLastNode()
if result_code == 0:
    print(f"自上次节点以来的行驶距离: {distance:.2f}米")
```

2.5.8 触发已注册的任务链 (triggerTaskChain)

```
def triggerTaskChain(self, taskChainId: str, loopCount: int = 1,
    variableMap: Optional[List[Dict[str, Any]]] = None) -> int:
```

参数说明:

- **taskChainId: str** - 任务链 ID, 用于指定要触发的目标任务链
- **loopCount: int** - 循环执行次数, 默认为 1。传入-1 表示无限循环。
- **variableMap: Optional[List[Dict[str, Any]]]** - 变量映射列表, 格式为[{"key": 变量名, "value": 变量值}, ...]。不传则使用默认的变量配置。

返回值:

- **int** - 结果码, 0 表示成功, 非 0 表示失败

使用示例:

```
# 触发 ID 为"task_chain_001"的任务链, 执行1 次
result_code = sdk.triggerTaskChain("task_chain_001")
if result_code == 0:
    print("任务链触发成功")
else:
    print(f"任务链触发失败, 错误码: {result_code}")

# 触发任务链, 循环执行3 次
result_code = sdk.triggerTaskChain("task_chain_001", loopCount=3)
if result_code == 0:
```

```
print("循环任务链触发成功")

# 触发任务链, 无限循环执行
result_code = sdk.triggerTaskChain("task_chain_001", loopCount=-1)
if result_code == 0:
    print("无限循环任务链触发成功")

# 定义变量映射参数
custom_vars = [
    {"key": "speed", "value": 50},
    {"key": "target_pos", "value": "point_A"}
]

# 触发任务链, 执行1次并传入自定义变量
result_code = sdk.triggerTaskChain("task_chain_001",
variableMap=custom_vars)
if result_code == 0:
    print("带变量的任务链触发成功")

# 无限循环 + 自定义变量
result_code = sdk.triggerTaskChain(
    taskChainId="task_chain_001",
    loopCount=-1,
    variableMap=[{"key": "mode", "value": "auto"}]
)
```

2.5.9 重新加载任务链定义文件 (reloadTaskChainFolder)

```
def reloadTaskChainFolder(self) -> int:
```

返回值:

- **int** - 结果码, 0 表示成功, 非 0 表示失败

使用示例:

```
# 重新加载任务链目录下的所有定义文件
result_code = sdk.reloadTaskChainFolder()
if result_code == 0:
    print("任务链目录重新加载成功")
else:
    print(f"任务链目录重新加载失败，错误码：{result_code}")
```

2.6 即时动作

2.6.1 重新定位 (reposition)

```
def reposition(self, x: float, y: float, theta: float, auto_mode: bool
= False) -> int:
```

参数说明:

- **x: float** - 目标 X 坐标
- **y: float** - 目标 Y 坐标
- **theta: float** - 目标角度 (弧度)
- **auto_mode: bool** - 自动重定位模式，默认 False

返回值:

- **int** - 结果码，0 表示成功，非 0 表示失败

使用示例:

```
# 重新定位到指定位置
result = sdk.reposition(10.5, 5.2, 1.57) # x=10.5m, y=5.2m, theta=90 度
if result == 0:
    print("重新定位成功")
else:
    print(f"重新定位失败，错误码：{result}")
```

```
# 自动重定位模式
result = sdk.reposition(0, 0, 0, auto_mode=True)
```

2.6.2 AGV 旋转 (vehicleRotation)

```
def vehicleRotation(self, theta: float, coordinate: str = "MAP") -> int:
```

参数说明:

- **theta: float** - 目标角度 (弧度)
- **coordinate: str** - 坐标系, "MAP" (地图坐标系) 或 "ROBOT" (机器人坐标系), 默认 "MAP"

返回值:

- **int** - 结果码, 0 表示成功, 非 0 表示失败

使用示例:

```
# 在地图坐标系下旋转90度
result = sdk.vehicleRotation(1.57, "MAP")
if result == 0:
    print("车辆旋转成功")

# 在机器人坐标系下旋转
result = sdk.vehicleRotation(3.14, "ROBOT")
```

2.6.3 获取 AGV 位置 (getAgvPosition)

```
def getAgvPosition(self) -> tuple[int, Position]:
```

返回值:

- **tuple[int, Position]** - (结果码, 位置信息对象)
 - 结果码为 0 表示成功, 非 0 表示失败
 - 位置信息对象详见第 4 章 数据类型

使用示例:

```
result_code, position = sdk.getAgvPosition()
if result_code == 0:
    print(f"AGV 位置: X={position.x:.2f}, Y={position.y:.2f}, 角度={position.theta:.2f}")
    print(f"定位状态: {position.status}, 定位质量: {position.localizationScore}")
```

2.6.4 获取 AGV 速度 (getVelocity)

```
def getVelocity(self) -> tuple[int, Velocity]:
```

返回值:

- **tuple[int, Velocity]** - (结果码, 速度信息对象)
 - 结果码为 0 表示成功, 非 0 表示失败
 - 速度信息对象详见第 4 章 数据类型

使用示例:

```
result_code, velocity = sdk.getVelocity()
if result_code == 0:
    print(f"速度: X={velocity.vx:.2f}m/s, Y={velocity.vy:.2f}m/s, 角速度={velocity.omega:.2f}rad/s")
```

2.6.5 取消任务 (cancelTask)

```
def cancelTask(self) -> int:
```

返回值:

- **int** - 结果码, 0 表示成功, 非 0 表示失败

使用示例:

```
result = sdk.cancelTask()
if result == 0:
    print("任务取消成功")
else:
    print(f"任务取消失败, 错误码: {result}")
```

2.6.6 暂停任务 (pauseTask)

```
def pauseTask(self) -> int:
```

返回值:

- **int** - 结果码, 0 表示成功, 非 0 表示失败

使用示例:

```
result = sdk.pauseTask()
if result == 0:
    print("任务暂停成功")
```

2.6.7 恢复任务 (restoreTask)

```
def restoreTask(self) -> int:
```

返回值:

- **int** - 结果码, 0 表示成功, 非 0 表示失败

使用示例:

```
result = sdk.restoreTask()
if result == 0:
    print("任务恢复成功")
```

2.6.8 启用指定地图(enableMap)

```
def enableMap(self, mapId: str) -> int:
```

参数说明:

- **mapId: str** - 地图 ID

返回值:

- **int** - 结果码, 0 表示成功, 非 0 表示失败

使用示例:

```
result = sdk.enableMap("map_001")
if result == 0:
    print("地图启用成功")
```

2.6.9 获取地图信息 (getMap)

```
def getMap(self) -> tuple[int, MapInfo]:
```

返回值:

- **tuple[int, MapInfo]** - (结果码, 地图信息对象)
 - 结果码为 0 表示成功, 非 0 表示失败
 - 地图信息对象详见第 4 章 数据类型

使用示例:

```
result_code, map_info = sdk.getMap()
if result_code == 0:
    print(f"地图 ID: {map_info.mapId}")
    print(f"定位图 ID: {map_info.locationMapId}")
    print(f"地图版本: {map_info.mapVersion}")
    print(f"地图描述: {map_info.mapDescription}")
```

2.6.10 下载地图 (downloadMap)

```
def downloadMap(self, map_md5_infos: StringMap) -> int:
```

参数说明:

- **map_md5_infos: StringMap** - 该接口用于从地图服务器获取最新地图到机器人, 请先从地图服务器获取要下载的地图的 MD5 信息字典

返回值:

- **int** - 结果码, 0 表示成功, 非 0 表示失败

使用示例:

```
map_info = pypilot.StringMap()
map_info["map_001"] = "a1b2c3d4e5f6"
result = sdk.downloadMap(map_info)
if result == 0:
    print("地图下载成功")
```

2.6.11 软急停控制 (softStop)

```
def softStop(self, operate: str) -> int:
```

参数说明:

- **operate: str** - OPEN=开启急停, CLOSE=解除急停

返回值:

- **int** - 结果码, 0 表示成功, 非 0 表示失败

使用示例:

```
# 开启软急停
sdk.softStop("OPEN")
# 解除软急停
sdk.softStop("CLOSE")
```

2.6.12 充电控制 (charge)

```
def charge(self, chargeMode: str, time: int, batteryLevel: int) -> int:
```

参数说明:

- **chargeMode: str** - 充电模式
 - SPECIFY_TIME: 指定时间, time 参数生效
 - SPECIFY_BATTERY_LEVEL: 指定电量, batteryLevel 参数生效
- **time: int** - 充电时间 (分钟)
- **batteryLevel: int** - 目标电量级别 (%)

返回值:

- **int** - 结果码, 0 表示成功, 非 0 表示失败

使用示例:

```
result = sdk.charge("SPECIFY_TIME", 60, 0) # 自动充电 1 小时
if result == 0:
    print("充电控制设置成功")

result = sdk.charge("SPECIFY_BATTERY_LEVEL ", 0, 80) # 自动充电至 80%
if result == 0:
```

```
print("充电控制设置成功")
```

2.6.13 精准调整 (accurateAdjust)

```
def accurateAdjust(self, featureType: str, x: float, y: float, theta: float, qrAdjustAngleLimit: float, qrAdjustDistanceLimit: float, dockId: int, oobjectIdId: int) -> int:
```

参数说明:

该接口用于在特征对接时发现对接后位置 and 实际期望位置有微量偏差后，手动给该对接位置设置一个偏移值，精准调整对接位置。

- **featureType: str** - 特征类型
 - QR_UP: 上二维码
 - QR_DOWN: 下二维码
 - QR_LEFT: 左二维码
 - QR_RIGHT: 右二维码
 - QR_SIDE: 侧面二维码
 - LASER_SIDE: 侧面激光
 - QR_ADJUST: 上二维码 (高速)
 - LINE_STRAIGHT: 单悬臂
 - DUAL_CAMERA: 双相机
 - REFLECTOR_ADJUST: 反光贴
- **x: float** - 偏移 X 坐标

- **y: float** - 偏移 Y 坐标
- **theta: float** - 偏移角度
- **qrAdjustAngleLimit: float** - QR 调整角度限制, 仅特征类型为 QR_UP, QR_DOWN, QR_LEFT, QR_RIGHT 时有效
- **qrAdjustDistanceLimit: float** - QR 调整距离限制, 仅特征类型为 QR_UP, QR_DOWN, QR_LEFT, QR_RIGHT 时有效
- **dockId: int** - 对接工位 ID
- **oobjectId: int** - 反光贴模板 ID, 需要先在 Toolkit 中配置, 仅特征类型为 REFLECTOR_ADJUST 时生效

返回值:

- **int** - 结果码, 0 表示成功, 非 0 表示失败

使用示例:

```
result = sdk.accurateAdjust("QR_UP", 10.0, 5.0, 0.0, 0.1, 0.05, 1,
1001)
if result == 0:
    print("精准调整成功")
```

2.6.14 对接操作 (docking)

```
def docking(self, featureType: str, isCharge: bool, x: float, y: float,
theta: float, direction: str, dockId: int, oobjectId: int,
cameraAvoidance: bool, start_x: float, start_y: float, start_theta:
float, end_x: float, end_y: float, end_theta: float) -> int:
```

参数说明:

- **featureType: str** - 特征类型
 - UNDOCKING: 脱离对接
 - QR_DOCKING: 二维码对接
 - SHELF_LEGS_DOCKING: 货架腿对接
 - V_SHAPED_PLATE_DOCKING: V 形板对接
 - REFLECTORS_DOCKING: 反光贴/反光板对接
- **isCharge: bool** - 是否为充电对接
- **x: float** - 目标 X 坐标
- **y: float** - 目标 Y 坐标
- **theta: float** - 目标角度
- **direction: str** - 方向
 - FRONT: 车头对接
 - BACK: 车尾对接
- **dockId: int** - 对接工位 ID
- **objectId: int** - 反光贴或货架腿的模板 ID
- **cameraAvoidance: bool** - 是否开启相机避障
- **start_x: float** - 对接路径起始点 X 坐标
- **start_y: float** - 对接路径起始点 Y 坐标
- **start_theta: float** - 对接路径起始点角度
- **end_x: float** - 对接路径结束点 X 坐标
- **end_y: float** - 对接路径结束点 Y 坐标
- **end_theta: float** - 对接路径结束点角度

返回值:

- **int** – 结果码, 0 表示成功, 非 0 表示失败

使用示例:

```
result = sdk.docking("QR_DOCKING", True, 10.0, 5.0, 0.0, "FRONT", 1,
1001, True, 9.0, 4.0, 0.0, 11.0, 6.0, 0.0)
if result == 0:
    print("对接操作成功")
```

2.6.15 清除错误 (clearError)

```
def clearError(self, type: str, deviceId: str = "") -> int:
```

参数说明:

- **type: str** – 清错类型
 - DOCKING: 对接错误
 - SCRIPT: 脚本错误
- **deviceId: str** – 清错的设备 ID, 当前版本无需指定, 留空即可。

返回值:

- **int** – 结果码, 0 表示成功, 非 0 表示失败

使用示例:

```
result = sdk.clearError("DOCKING")
if result == 0:
    print("对接错误已清除")
```

2.6.16 播放音频 (audioPlay)

```
def audioPlay(self, playTrigger: str, audioName: str, audioVolume: int,
playCount: int) -> int:
```

参数说明:

- **playTrigger: str** - 播放触发器
 - PLAY: 开始播放
 - STOP: 停止播放
- **audioName: str** - 音频名称
- **audioVolume: int** - 音频音量
- **playCount: int** - 播放次数, 0 表示无限循环播放。

返回值:

- **int** - 结果码, 0 表示成功, 非 0 表示失败

使用示例:

```
result = sdk.audioPlay("PLAY", "alert.wav", 80, 1)
if result == 0:
    print("音频播放成功")
```

2.6.17 读取车辆寄存器 (readVehicleRegister)



NOTE

用户使用寄存器相关接口前应先系统学习对应 MODBUS 寄存器相关的知识，后续接口不针对寄存器相关概念做详细解释。

```
def readVehicleRegister(self, code: int, slavedId: int, address: int)
-> tuple[int, int]:
```

参数说明:

- **code: int** - 寄存器功能码
- **slavedId: int** - 从站设备 ID
- **address: int** - 寄存器地址

返回值:

- **tuple[int, int]** - (结果码, 读取值)
 - 结果码为 0 表示成功, 非 0 表示失败
 - 读取值为对应寄存器的实际值

使用示例:

```
result_code, value = sdk.readVehicleRegister(1, 2, 100)
if result_code == 0:
    print(f"寄存器值: {value}")
```

2.6.18 写入车辆寄存器 (writeVehicleRegister)

```
def writeVehicleRegister(self, code: int, slavedId: int, address: int,
value: int) -> int:
```

参数说明:

- **code: int** - 寄存器功能码
- **slavedId: int** - 从站设备 ID
- **address: int** - 寄存器地址
- **value: int** - 要写入的值

返回值:

- **int** - 结果码, 0 表示成功, 非 0 表示失败

使用示例:

```
result = sdk.writeVehicleRegister(1, 2, 100, 255)
if result == 0:
    print("寄存器写入成功")
```

2.6.19 读取外部寄存器 (readExternalRegister)

```
def readExternalRegister(self, ip: str, port: int, code: int, slaveId:
int, address: int) -> tuple[int, int]:
```

参数说明:

- **ip: str** - 外部设备 IP 地址

- **port: int** - 端口号
- **code: int** - 寄存器功能码
- **slavedId: int** - 从站设备 ID
- **address: int** - 寄存器地址

返回值:

- **tuple[int, int]** - (结果码, 读取值)
 - 结果码为 0 表示成功, 非 0 表示失败
 - 读取值为对应寄存器的实际值

使用示例:

```
result, value = sdk.readExternalRegister("192.168.1.100", 502, 3, 1, 0)
if result_code == 0:
    print(f"寄存器值: {value}")
```

2.6.20 写入外部寄存器 (writeExternalRegister)

```
def writeExternalRegister(self, ip: str, port: int, code: int, slaveId:
int, address: int, value: int) -> int:
```

参数说明:

- **ip: str** - 外部设备 IP 地址
- **port: int** - 端口号
- **code: int** - 寄存器功能码
- **slavedId: int** - 从站设备 ID

- **address: int** - 寄存器地址
- **value: int** - 要写入的值

返回值:

- **int** - 结果码, 0 表示成功, 非 0 表示失败

使用示例:

```
result = sdk.writeExternalRegister("192.168.1.100", 502, 3, 1, 0)
if result == 0:
    print("寄存器写入成功")
```

2.6.21 校验外部寄存器 (checkExternalRegister)

```
def checkExternalRegister(self, ip: str, port: int, code: int, slaveId:
int, address: int, normalExit: int, failedExit: int = -1, timeout: int
= -1) -> tuple[int, bool]:
```

参数说明:

- **ip: str** - 外部设备 IP 地址
- **port: int** - 端口号
- **code: int** - 寄存器功能码
- **slavedId: int** - 从站设备 ID
- **address: int** - 寄存器地址
- **normalExit: int** - 正常退出码
- **failedExit: int** - 失败退出码, 默认-1

- **timeout: int** - 超时时间，默认-1，表示永不超时

返回值:

- **tuple[int, bool]** - (结果码, 校验结果)
 - 结果码为 0 表示成功，非 0 表示失败
 - 校验结果为 True 表示校验成功，False 表示校验失败

使用示例:

```
result_code, ok = sdk.checkExternalRegister("192.168.1.100", 502, 3, 1,
100, 1)
if result_code == 0 and ok:
    print("外部寄存器校验成功")
```

2.6.22 切换避障场集 (switchAvoidanceFieldSet)

```
def switchAvoidanceFieldSet(self, fieldSet: int) -> int:
```

参数说明:

- **fieldSet: int** - 场集编号

返回值:

- **int** - 结果码，0 表示成功，非 0 表示失败

使用示例:

```
result = sdk.switchAvoidanceFieldSet(2)
if result == 0:
    print("避障场集切换成功")
```

2.6.23 切换速度避障模式 (switchSpeedAvoidanceMode)

```
def switchSpeedAvoidanceMode(self, speedMode: str) -> int:
```

参数说明:

- **speedMode: str** - 速度模式
 - **DEFAULT**: 默认速度模式
 - **USER**: 用户速度模式, 需要先在 Toolkit 中进行配置

返回值:

- **int** - 结果码, 0 表示成功, 非 0 表示失败

使用示例:

```
result = sdk.switchSpeedAvoidanceMode(DEFAULT)
if result == 0:
    print("速度避障模式切换成功")
```

2.7 机械臂



NOTE

用户使用机械臂相关接口前应先系统学习对应机械臂相关的知识, 后述接口不针对机械臂相关概念做详细解释。

2.7.1 机械臂关节坐标说明

隙锋机器人共配备了三个机械臂：左臂，右臂，腰腿部（以下简称为腰）。

- 其中左臂和右臂为 7 轴机械臂，使用关节坐标表达时为[J1, J2, J3, J4, J5, J6, J7]，顺序为**从肩到手**。
- 腰为 3 轴机械臂，使用关节坐标表达时为[J1, J2, 0, J4]，顺序为**从下往上**（底盘到躯干），第一个关节为 J1，第二个关节为 J2，第三个关节为 J4。

2.7.2 初始化机械臂数据连接 (armInitData)

```
def armInitData(self, localIp: str, armIp: str, port: int) -> int:
```

参数说明：

- **localIp: str** - 本地 IP 地址，用于接受文件
- **armIp: str** - 机械臂控制器 IP 地址
- **port: int** - 端口号

返回值：

- **int** - 结果码，0 表示成功，非 0 表示失败

使用示例：

```
sdk.armInitData("192.168.1.100", "192.168.1.200", 8080)
```

2.7.3 获取机械臂控制器连接状态 (armGetLinkStatus)

```
def armGetLinkStatus() -> tuple[int, bool]:
```

返回值:

- **tuple[int, bool]** - (结果码, 连接状态)
 - 结果码为 0 表示成功, 非 0 表示失败
 - 连接状态为 True 表示连接正常, False 表示连接异常

使用示例:

```
result_code, status = sdk.armGetLinkStatus()
if result_code == 0:
    if status:
        print("连接正常")
    else:
        print("连接异常, 请检查网络")
```

2.7.4 脱离示教器 (armDetachHMI)

```
def armDetachHmi() -> int:
```

返回值:

- **int** - 结果码, 0 表示成功, 非 0 表示失败

使用示例:

```
# 如需拔掉机械臂示教器, 请先调用该接口
```

```
sdk.armDetachHmi() # 调用后需要拔掉示教器
```

2.7.5 获取脱离示教器状态 (armGetDetachHMIStatus)

```
def armGetDetachHMIStatus(self) -> tuple[int, bool]:
```

返回值:

- **tuple[int, bool]** - (结果码, 校验结果)
 - 结果码为 0 表示成功, 非 0 表示失败
 - 校验结果为 True 表示已脱离示教器, False 表示未脱离示教器

使用示例:

```
result_code, ok = sdk.armGetDetachHMIStatus()
if result_code == 0 and ok:
    print("已脱离示教器")
```

2.7.6 获取伺服 OP 状态 (armServolsOpOrNot)

```
def armServoIsOpOrNot() -> tuple[int, bool]:
```

返回值:

- **tuple[int, bool]** - (结果码, 伺服 OP 状态)
 - 结果码为 0 表示成功, 非 0 表示失败

- 伺服 OP 状态为 True 表示伺服 OP 正常，False 表示伺服未 OP

使用示例：

```
result, status = sdk.armServoIsOpOrNot()
if result == 0 and status:
    print("伺服 OP 正常")
```

2.7.7 获取报警信息 (armIsAlarming)

```
def armIsAlarming() -> tuple[int, StringVector]:
```

返回值：

- **tuple[int, StringVector]** - (结果码, 报警信息列表)
 - **结果码**：0 表示有报警，非 0 表示无报警或接口调用失败
 - **报警信息列表**：列表中每一个字符串描述一个报警

使用示例：

```
is_alarm, alarm_info = sdk.armIsAlarming()
if is_alarm:
    print(f"报警信息: {alarm_info}")
```

2.7.8 清除报警 (armClearAlarm)

```
def armClearAlarm() -> int:
```

返回值:

- **int** - 结果码, 0 表示成功, 非 0 表示失败

使用示例:

```
sdk.armClearAlarm()
```

2.7.9 机械臂使能控制 (armRobotEnableOrNot)

```
def armRobotEnableOrNot(enable: bool) -> int:
```

参数说明:

- **enable: bool** - True 表示上使能, False 表示下使能

返回值:

- **int** - 结果码, 0 表示成功, 非 0 表示失败

使用示例:

```
sdk.armRobotEnableOrNot(True)    # 上使能  
sdk.armRobotEnableOrNot(False)  # 下使能
```

2.7.10 获取使能状态 (armGetRobotEnableStatus)

```
def armGetRobotEnableStatus() -> tuple[int, bool]:
```

返回值:

- **tuple[int, bool]** - (结果码, 使能状态)
 - 结果码为 0 表示成功, 非 0 表示失败
 - 使能状态为 True 表示已使能, False 表示未使能

使用示例:

```
result, EnableStatus = sdk.armGetRobotEnableStatus()
if EnableStatus:
    print("机械臂已使能")
```

2.7.11 单个机械臂使能控制 (armSetSingleRobotEnable)

```
def armSetSingleRobotEnable(robotId: int, enable: bool) -> int:
```

参数说明:

- **robotId: int** - 机械臂 ID (1: 左臂, 2: 右臂, 3: 腰)
- **enable: bool** - True 表示上使能, False 表示下使能

返回值:

- **int** - 结果码, 0 表示成功, 非 0 表示失败

使用示例:

```
sdk.armSetSingleRobotEnable(1, True) # 机械臂1 上使能
```

2.7.12 获取单个机械臂使能状态

(armGetSingleRobotEnableStatus)

```
def armGetSingleRobotEnableStatus(robotId: int) -> tuple[int, bool]:
```

参数说明:

- **robotId: int** - 机械臂 ID (1: 左臂, 2: 右臂, 3: 腰)

返回值:

- **tuple[int, bool]** - (结果码, 使能状态)
 - 结果码为 0 表示成功, 非 0 表示失败
 - 使能状态为 True 表示已使能, False 表示未使能

使用示例:

```
result, EnableStatus = sdk.armGetSingleRobotEnableStatus(1)
if EnableStatus:
    print("机械臂 1 已使能")
```

2.7.13 设置全局速度 (armSetGlobalSpeed)

```
def armSetGlobalSpeed(speed: float) -> int:
```

参数说明:

- **speed: float** - 全局速度, 范围 0-100 (%)

返回值:

- **int** - 结果码, 0 表示成功, 非 0 表示失败

使用示例:

```
sdk.armSetGlobalSpeed(50.0) # 设置全局速度为 50%
```

2.7.14 获取全局速度 (armGetGlobalSpeed)

```
def armGetGlobalSpeed() -> tuple[int, float]:
```

返回值:

- **tuple[int, float]** - (结果码, 当前全局速度)
 - 结果码为 0 表示成功, 非 0 表示失败
 - 当前全局速度范围 0-100 (%)

使用示例:

```
result, speed = sdk.armGetGlobalSpeed()  
print(f"当前全局速度: {speed}%")
```

2.7.15 设置报警时间伺服是否下使能

(armSetRobotAlarmNotServoOff)

```
def armSetRobotAlarmNotServoOff(needNotOff: bool) -> int:
```

参数说明:

- **needNotOff: bool** - True 表示报警时不下使能, False 表示下使能

返回值:

- **int** - 0 表示设置成功, 非 0 表示设置失败

使用示例:

```
sdk.armSetRobotAlarmNotServoOff(True) # 报警时不下使能
```

2.7.16 获取报警时伺服是否下使能

(armGetRobotAlarmNotServoOff)

```
def armGetRobotAlarmNotServoOff() -> tuple[int, bool]:
```

返回值:

- **tuple[int, bool]** - (结果码, 是否下使能)
 - 结果码为 0 表示接口调用成功, 非 0 表示接口调用失败
 - 是否下使能为 True 表示不下使能, False 表示下使能

使用示例:

```
result, servoOff = sdk.armGetRobotAlarmNotServoOff()
if servoOff:
    print("报警时伺服不会下使能")
```

2.7.17 设置报警提示配置 (armSetAlarmNotPrompt)

```
def armSetAlarmNotPrompt(self, flag: int) -> int:
```

参数说明:

- **flag: int** – 报警提示标志位 (二进制位控制)
 - **bit0**: 1 表示正负极限不报警, 0 表示正常报警。
 - **bit1**: 1 表示路径规划失败不报警, 0 表示正常报警。

返回值:

- **int** – 结果码, 0 表示成功, 非 0 表示失败

使用示例:

```
# 设置: 正负极限不报警 + 路径规划失败不报警 (flag = 0b11 = 3)
result = sdk.armSetAlarmNotPrompt(3)
if result == 0:
    print("告警不提示标志设置成功")
```

2.7.18 获取报警提示配置 (armGetAlarmNotPrompt)

```
def armGetAlarmNotPrompt(self) -> tuple[int, int]:
```

返回值:

- **tuple[int, int]** – (结果码, 报警提示标志位)
 - 结果码为 0 表示成功, 非 0 表示失败

- 报警提示标志位含义见设置报警提示配置 (armSetAlarmNotPrompt)

使用示例:

```
result_code, alarm_flag = sdk.armGetAlarmNotPrompt()
if result_code == 0:
    print(f"当前告警不提示标志位: {alarm_flag}")
```

2.7.19 腰部安全姿态约束 (armSetWaistSafePose)

```
def armSetWaistSafePose(self, angle: float) -> int:
```

参数说明:

- **angle: float** – 腰部安全约束角度, 约束规则: $\text{abs}(-J1+J2+J3) < \text{angle}$

返回值:

- **int** – 结果码, 0 表示成功, 非 0 表示失败

使用示例:

```
# 设置腰部安全约束角度为 30.0
result = sdk.armSetWaistSafePose(30.0)
if result == 0:
    print("机械臂腰部安全姿态约束设置成功")
```

2.7.20 获取腰部安全姿态约束 (armGetWaistSafePose)

```
def armGetWaistSafePose(self) -> Tuple[int, float]:
```

返回值:

- **tuple[int, float]** - (结果码, 约束角度)
 - 结果码为 0 表示成功, 非 0 表示失败
 - 约束角度为当前生效值。

使用示例:

```
result_code, angle = sdk.armGetWaistSafePose()
if result_code == 0:
    print(f"当前腰部安全约束角度: {angle}")
```

2.7.21 设置机械臂保护状态 (armSetRobotProtectStatus)

```
def armSetRobotProtectStatus(robotId: int, protect: bool) -> int:
```

参数说明:

- **robotId: int** - 机械臂 ID (1: 左臂, 2: 右臂, 3: 腰), 传入-1 操作所有机械臂
- **protect: bool** - True 表示保护状态, False 表示非保护状态

当开启保护状态时, 系统会实时监测指令参数的异常突变 (例如单次指令值为 100, 下一次突变为 10000)。如果检测到此类不符合正常运动学约束的数值跳变, 系统将判定为异常指令并自动触发保护, 防止机械臂执行潜在的危险动作

在透传控制模式 (直接下发底层控制指令) 下, 用户可能需要自由传入任意数值进行自定义轨迹规划。此时为避免正常的透传指令被误判为异常, 应当关闭保护状态

返回值:

- **int** - 结果码, 0 表示成功, 非 0 表示失败

使用示例:

```
sdk.armSetRobotProtectStatus(1, False) # 设置机械臂 1 为非保护状态
```

2.7.22 获取机械臂保护状态 (armGetRobotProtectStatus)

```
def armGetRobotProtectStatus(robotId: int) -> tuple[int, bool]:
```

参数说明:

- **robotId: int** - 机械臂 ID (1: 左臂, 2: 右臂, 3: 腰)

返回值:

- **tuple[int, bool]** - (结果码, 保护状态)
 - 结果码为 0 表示接口调用成功, 非 0 表示接口调用失败
 - 保护状态为 True 表示保护状态, False 表示非保护状态

使用示例:

```
result, protectStatus= sdk.armGetRobotProtectStatus(1)
if protectStatus == 0:
    print("机械臂 1 处于保护状态")
```

2.7.23 打开串口 (armOpenCom)

```
def armOpenCom(self, com: str, baudrate: int = 115200, databits: int = 8, parity: int = 0, stopbits: int = 1) -> int:
```

参数说明:

- **com: str** - 串口名称, 如 "COM1" 或 "COM2"
- **baudrate: int** - 波特率, 常用值: 9600, 19200, 38400, 115200。默认 115200
- **databits: int** - 数据位, 默认 8
- **parity: int** - 校验位, 0=无校验, 1=奇校验, 2=偶校验, 默认 0
- **stopbits: int** - 停止位, 默认 1

返回值:

- **int** - 0 表示操作成功, 非 0 表示失败

使用示例:

```
from pypilot import PilotSDK

sdk = PilotSDK("192.168.1.100")
sdk.start()

# 打开 COM1, 使用默认参数 (115200, 8, 0, 1)
result = sdk.armOpenCom("COM1")
if result == 0:
    print("串口打开成功")
else:
    print("串口打开失败")

# 打开 COM2, 指定波特率为 9600
result = sdk.armOpenCom("COM2", baudrate=9600)
if result == 0:
```

```
print("串口打开成功")
```

2.7.24 读串口 (armReadCom)

```
def armReadCom(self, com: str, buffer_len: int) -> tuple[int, bytes]:
```

参数说明:

- **com: str** - 串口名称, 需要先通过 armOpenCom 接口打开
- **buffer_len: int** - 读取缓冲区长度, 建议设置为期望读取的最大字节数

返回值:

- **tuple[int, bytes]** - (字节数, 读取数据)
 - 字节数: 实际读取的字节数, 如果为 0 表示没有数据或读取失败
 - 读取数据: 实际读取到的数据 (原始二进制)

使用示例:

```
# 读取最多 256 字节的数据
read_count, data = sdk.armReadCom("COM1", 256)
if read_count > 0:
    print(f"读取到 {read_count} 字节数据: {data}")
```

2.7.25 写串口 (armWriteCom)

```
def armWriteCom(self, com: str, buffer: str) -> int:
```

参数说明:

- **com: str** - 串口名称, 需要先通过 armOpenCom 接口打开
- **buffer: str/bytes** - 要写入的数据, 可以是字符串或字节对象

返回值:

- **int** - 0 表示操作成功, 非 0 表示失败

使用示例:

```
# 写入字符串数据
result = sdk.armWriteCom("COM1", "Hello, World!")
if result == 0:
    print("数据写入成功")
else:
    print("数据写入失败")

# 写入字节数据 (推荐方式)
data = b'\x01\x02\x03\x04'
result = sdk.armWriteCom("COM1", data)
if result == 0:
    print("字节数据写入成功")

# 写入十六进制字符串
hex_data = bytes.fromhex("01020304")
result = sdk.armWriteCom("COM1", hex_data)
```

2.7.26 Modbus 控制 (armModbusControl)

```
def armModbusControl(self, cmd: int, modbusData: ModbusRtuData) -> int:
```

参数说明:

- **cmd: intr** - Modbus 命令类型, 使用 ModbusCommandType 枚举值:
 - ModbusCommandType.CMD_READ_INPUT_REGISTER: 对应数值 0x04, 读单个输入寄存器
 - ModbusCommandType.CMD_WRITE_MULTIPLE_REGISTERS: 对应数值 0x10 写多个寄存器
- **modbusData: ModbusRtuData** - Modbus RTU 数据结构, 字段如下:
 - **name: str**: 串口名称, 如 "COM1" 或 "COM2"
 - **slaveid: int**: 从站 ID (设备地址)
 - **addr: int**: 首个寄存器地址
 - **amount: int**: 数据长度 (寄存器数量, 不是字节数)
 - **write_value: int**: 写单个寄存器时使用的值 (uint16_t)
 - **writeValues: UInt16Vector**: 写多个寄存器时使用的值列表 (List[int])

返回值:

- **int** - 0 表示操作成功, 非 0 表示失败

使用示例:

示例 1: 读取输入寄存器

```
from pypilot import PilotSDK, ModbusCommandType, ModbusRtuData

sdk = PilotSDK("192.168.1.100")
sdk.start()

# 先打开串口
sdk.armOpenCom("COM1", baudrate=9600)

# 创建 Modbus 数据对象
modbus_data = ModbusRtuData()
modbus_data.name = "COM1"
```

```
modbus_data.slaveid = 1      # 从站 ID 为 1
modbus_data.addr = 500      # 寄存器起始地址
modbus_data.amount = 1      # 读取 1 个寄存器

# 执行读取操作
result =
sdk.armModbusControl(ModbusCommandType.CMD_READ_INPUT_REGISTER,
modbus_data)
if result == 0:
    print("Modbus 读取命令发送成功")
    # 注意: 实际读取的数据需要通过回调函数获取
else:
    print("Modbus 读取命令发送失败")
```

示例 2: 写入多个寄存器

```
# 创建 Modbus 数据对象
modbus_data = ModbusRtuData()
modbus_data.name = "COM1"
modbus_data.slaveid = 1      # 从站 ID 为 1
modbus_data.addr = 500      # 寄存器起始地址
modbus_data.amount = 3      # 写入 3 个寄存器

# 设置要写入的值 (多个寄存器)
modbus_data.writeValues = [100, 200, 300] # 写入 3 个值

# 执行写入操作
result =
sdk.armModbusControl(ModbusCommandType.CMD_WRITE_MULTIPLE_REGISTERS,
modbus_data)
if result == 0:
    print("Modbus 写入命令发送成功")
else:
    print("Modbus 写入命令发送失败")
```

示例 3: 写入单个寄存器

```

# 创建 Modbus 数据对象
modbus_data = ModbusRtuData()
modbus_data.name = "COM1"
modbus_data.slaveid = 1
modbus_data.addr = 500
modbus_data.amount = 1
modbus_data.write_value = 123 # 写入单个寄存器的值

# 执行写入操作（注意：写单个寄存器也使用 CMD_WRITE_MULTIPLE_REGISTERS）
result =
sdk.armModbusControl(ModbusCommandType.CMD_WRITE_MULTIPLE_REGISTERS,
modbus_data)

```

注意事项:

1. 使用 armModbusControl 前，需要先通过 armOpenCom 打开对应的串口。
2. 读取操作的结果需要通过注册的回调函数获取（需要先注册 Modbus 回调）。
3. 写入多个寄存器时，writeValues 列表的长度应该等于 amount 的值。
4. 串口参数（波特率、数据位等）需要在 armOpenCom 时正确配置，确保与 Modbus 设备匹配。

2.7.27 机械臂直线运动 (armMoveWorlds)

```

def armMoveWorlds(robotId: int, worlds: FloatVector, interpolation_en:
bool = False, needCallback: bool = False, delay_in_ms: int = 0) -> int:

```

参数说明:

- **robotId: int** - 机械臂 ID (1: 左臂, 2: 右臂, 3: 腰)
- **worlds: FloatVector** - 目标点在世界坐标系中的坐标 [X, Y, Z, U, V, W]
- **interpolation_en: bool** - 是否启用平滑衔接, 默认 False
- **needCallback: bool** - 是否调用回调, 默认 False。回调函数用于通知运动是

否结束，详见第 3 章 回调函数

- **delay_in_ms: int** - 滞后执行时间(毫秒)，默认 0



NOTE

当前版本如需控制腰部机械臂，建议使用关节角度进行控制。如需使用世界坐标控制腰部，需要将 `pose[3] = 0.0`, `pose[4] = 0.0`, `pose[5] = 90` 再进行发送。同时腰部世界坐标只支持对 XYZUVW 中的 X 和 Z 进行更改。

返回值:

- **int** - 结果码，0 表示成功，非 0 表示失败

使用示例:

```
sdk.armMoveWorlds(1, [100.2, 200.5, 300.0, 0.0, 0.0, 0.0],
interpolation_en=True)
```

2.7.28 机械臂弧线运动 (armMoveWorldsCurve)

```
def armMoveWorldsCurve(robotId: int, midWorld: FloatVector, endWorld:
FloatVector, needCallback: bool = False, delay_in_ms: int = 0) -> int:
```

参数说明:

- **robotId: int** - 机械臂 ID (1: 左臂, 2: 右臂, 3: 腰)
- **midWorld: FloatVector** - 弧线中间点在世界坐标系中的坐标 [X, Y, Z, U, V, W]
- **endWorld: FloatVector** - 弧线结束点在世界坐标系中的坐标 [X, Y, Z, U, V, W]
- **needCallback: bool** - 是否调用回调，默认 False。回调函数用于通知运动是否结束，详见第 3 章 回调函数

- **delay_in_ms: int** - 滞后执行时间(毫秒), 默认 0

返回值:

- **int** - 结果码, 0 表示成功, 非 0 表示失败

使用示例:

```
sdk.armMoveWorldsCurve(1, [100, 200, 300, 0, 0, 0], [200, 300, 400, 0, 0, 0])
```

2.7.29 机械臂整圆运动 (armMoveWorldsRound)

```
def armMoveWorldsRound(robotId: int, midWorld: FloatVector, endWorld: l  
FloatVector, needCallback: bool = False, delay_in_ms: int = 0) -> int:
```

参数说明:

- **robotId: int** - 机械臂 ID (1: 左臂, 2: 右臂, 3: 腰)
- **midWorld: FloatVector** - 整圆中间点在世界坐标系中的坐标 [X, Y, Z, U, V, W]
- **endWorld: FloatVector** - 整圆结束点在世界坐标系中的坐标 [X, Y, Z, U, V, W]。与弧线运动不同, 机械臂到达结束点后不会停止运动, 而是通过整圆轨迹回到运动起始点。
- **needCallback: bool** - 是否调用回调, 默认 False。回调函数用于通知运动是否结束, 详见第 3 章 回调函数
- **delay_in_ms: int** - 滞后执行时间(毫秒), 默认 0

返回值:

- **int** - 结果码, 0 表示成功, 非 0 表示失败

使用示例:

```
sdk.armMoveWorldsRound(1, [100, 200, 300, 0, 0, 0], [200, 300, 400, 0, 0, 0])
```

2.7.30 移动到指定关节坐标位置 (armMoveJoints)

```
def armMoveJoints(robotId: int, joints: FloatVector, needCallback: bool = False, delay_in_ms: int = 0) -> int:
```

参数说明:

- **robotId: int** – 机械臂 ID (1: 左臂, 2: 右臂, 3: 腰)
- **joints: FloatVector** – 机械臂各关节的角度值, 详见 **2.7.1 机械臂关节坐标说明**
- **needCallback: bool** – 是否调用回调, 默认 False。回调函数用于通知运动是否结束, 详见**第 3 章 回调函数**
- **delay_in_ms: int** – 滞后执行时间(毫秒), 默认 0

返回值:

- **int** – 结果码, 0 表示成功, 非 0 表示失败

使用示例:

```
sdk.armMoveJoints(1, [0.0, 45.0, 90.0, 0.0, 0.0, 0.0])
```

2.7.31 角度转脉冲直接送伺服 (armPluseToServo)

```
def armPluseToServo(robotId: int, joints: FloatVector) -> int:
```

参数说明:

- **robotId: int** - 机械臂 ID (1: 左臂, 2: 右臂, 3: 腰)
- **joints: FloatVector** - 机械臂各关节的角度值, 详见 2.7.1 机械臂关节坐标说明

返回值:

- **int** - 结果码, 0 表示成功, 非 0 表示失败

使用示例:

```
# 配合 setRobotProtectStatus 使用, 用于关节摇操透传  
sdk.armPluseToServo(1, [0.0, 45.0, 90.0, 0.0, 0.0, 0.0])
```

2.7.32 世界坐标直接送伺服 (armWorldsToServo)

```
def armWorldsToServo(robotId: int, worlds: FloatVector) -> int:
```

参数说明:

- **robotId: int** - 机械臂 ID (1: 左臂, 2: 右臂, 3: 腰)
- **worlds: FloatVector** - 目标点在世界坐标系中的坐标 [X, Y, Z, U, V, W]

返回值:

- **int** - 结果码, 0 表示成功, 非 0 表示失败

使用示例:

```
# 配合 setRobotProtectStatus 使用, 用于末端摇操透传
sdk.armWorldsToServo(1, [100.0, 200.0, 300.0, 0.0, 0.0, 0.0])
```

2.7.33 获取当前关节坐标位置 (armGetJoints)

```
def armGetJoints(robotId: int, fb: bool = True) -> tuple[int,
FloatVector]:
```

参数说明:

- **robotId: int** - 机械臂 ID (1: 左臂, 2: 右臂, 3: 腰)
- **fb: bool** - True: 使用反馈值 (机械臂实时位置), False: 使用指令值 (机械臂目标位置)。默认 True

返回值:

- **tuple[int, FloatVector]** - (是否成功 (0 成功, 非 0 失败), 关节坐标列表)

使用示例:

```
success, joints = sdk.armGetJoints(1)
if success == 0:
    print(f"当前关节坐标: {joints}")
```

2.7.34 获取当前世界坐标位置 (armGetWorlds)

```
def armGetWorlds(robotId: int, fb: bool = True) -> tuple[int, FloatVector]:
```

参数说明:

- **robotId: int** - 机械臂 ID (1: 左臂, 2: 右臂, 3: 腰)
- **fb: bool** - True: 使用反馈值 (机械臂实时位置), False: 使用指令值 (机械臂目标位置)。默认 True

返回值:

- **tuple[int, FloatVector]** - (是否成功 (0 成功, 非 0 失败), 世界坐标列表)

使用示例:

```
success, worlds = sdk.armGetWorlds(1)
if success == 0:
    print(f"当前世界坐标: {worlds}")
```

2.7.35 获取当前规划速度 (armGetVelocity)

```
def armGetVelocity(robotId: int) -> tuple[int, float]:
```

参数说明:

- **robotId: int** - 机械臂 ID (1: 左臂, 2: 右臂, 3: 腰)

返回值:

返回值:

- **tuple[int, float]** - (是否成功 (0 成功, 非 0 失败), 当前规划速度)

使用示例:

```
result, velocity = sdk.armGetVelocity(1)
print(f"当前速度: {velocity}")
```

2.7.36 获取当前加速度 (armGetAcceleration)

```
def armGetAcceleration(robotId: int) -> tuple[int, float]:
```

参数说明:

- **robotId: int** - 机械臂 ID (1: 左臂, 2: 右臂, 3: 腰)

返回值:

- **tuple[int, float]** - (是否成功 (0 成功, 非 0 失败), 当前加速度)

使用示例:

```
accel = sdk.armGetAcceleration(1)
print(f"当前加速度: {accel}")
```

2.7.37 切换运行模式 (armSwitchRunMode)

```
def armSwitchRunMode(robotId: int, mode: int, axis: int = -1) -> int:
```

参数说明:

- **robotId: int** - 机械臂 ID (1: 左臂, 2: 右臂, 3: 腰)
- **mode: int** - 运行模式 (0: 位置, 1: 速度, 2: 力矩)
- **axis: int** - 轴索引 (从 0 开始), -1 表示所有轴

返回值:

- **int** - 是否切换成功 (0 成功, 非 0 失败)

使用示例:

```
sdk.armSwitchRunMode(1, 1, -1) # 切换所有轴到速度模式
```

2.7.38 获取轴运行模式 (armGetCurrentRunMode)

```
def armGetCurrentRunMode(robotId: int, axis: int) -> tuple[int, int]:
```

参数说明:

- **robotId: int** - 机械臂 ID (1: 左臂, 2: 右臂, 3: 腰)
- **axis: int** - 轴索引 (从 0 开始)

返回值:

- **tuple[int, int]** - (接口调用结果 (0 成功, 非 0 失败), 运行模式 (0: 位置, 1: 速度, 2: 力矩))

使用示例:

```
result, mode = sdk.armGetCurrentRunMode(1, 0)
print(f"轴 0 运行模式: {mode}")
```

2.7.39 获取反馈脉冲值 (armGetPulseFeedback)

```
def armGetPulseFeedback(robotId: int, axisId: int) -> tuple[int, int]:
```

参数说明:

- **robotId: int** - 机械臂 ID (1: 左臂, 2: 右臂, 3: 腰)
- **axisId: int** - 轴索引 (从 0 开始)

返回值:

- **tuple[int, int]** - (接口调用结果 (0 成功, 非 0 失败), 反馈脉冲值)

使用示例:

```
result, pulse = sdk.armGetPulseFeedback(1, 0)
print(f"轴 0 脉冲值: {pulse}")
```

2.7.40 获取机械臂力控扭矩 (armGetTorqueFeedbackVector)

```
def armGetTorqueFeedbackVector(robotId: int) -> tuple[int,
Int32Vector]:
```

参数说明:

- **robotId: int** - 机械臂 ID (1: 左臂, 2: 右臂, 3: 腰)

返回值:

- **tuple[int, Int32Vector]** - (接口调用结果 (0 成功, 非 0 失败), 扭矩反馈列表)

使用示例:

```
success, torques = sdk.armGetTorqueFeedbackVector(1)
if success == 0:
    for i in range(len(torques)):
        print(f"关节{i}扭矩反馈: {torques[i]}")
```

2.7.41 获取轴力控扭矩 (armGetTorqueFeedbackSingle)

```
def armGetTorqueFeedbackSingle(self, robotId: int, axisId: int) ->
tuple[int, int]:
```

参数说明:

- **robotId: int** - 机械臂 ID (1: 左臂, 2: 右臂, 3: 腰)
- **axisId: int** - 轴索引 (从 0 开始)

返回值:

- **tuple[int, int]** - (接口调用结果 (0 成功, 非 0 失败), 扭矩反馈)

使用示例:

```
success, torques = sdk.armGetTorqueFeedbackSingle(1,3)
if success == 0:
    print(f"机器 1 关节 3 扭矩反馈: {torques}")
```

2.7.42 获取速度反馈 (armGetSpeedFeedback)

```
def armGetSpeedFeedback(robotId: int) -> tuple[int, Int32Vector]:
```

参数说明:

- **robotId: int** - 机械臂 ID (1: 左臂, 2: 右臂, 3: 腰)

返回值:

- **tuple[int, Int32Vector]** - (接口调用结果 (0 成功, 非 0 失败), 速度反馈列表)

使用示例:

```
success, speeds = sdk.armGetSpeedFeedback(1)
if success:
    for i in range(len(speeds))
        print(f"关节{i}速度反馈: {speeds[i]}")
```

2.7.43 力矩模式控制 (armTorqueToServo)

```
def armTorqueToServo(robotId: int, torques: Int32Vector) -> int:
```

参数说明:

- **robotId: int** - 机械臂 ID (1: 左臂, 2: 右臂, 3: 腰)
- **torques: Int32Vector** - 力矩值 (单位根据电机定义)

返回值:

- **int** - 结果码, 0 表示成功, 非 0 表示失败

使用示例:

```
torques = pypilot.Int32Vector([100, 100, 100, 50, 50, 50])
sdk.armTorqueToServo(1, torques)
```

2.7.44 速度模式控制 (armSpeedToServo)

```
def armSpeedToServo(robotId: int, speeds: Int32Vector) -> int:
```

参数说明:

- **robotId: int** - 机械臂 ID (1: 左臂, 2: 右臂, 3: 腰)
- **speeds: Int32Vector** - 速度值 (单位根据电机定义)

返回值:

- **int** - 结果码, 0 表示成功, 非 0 表示失败

使用示例:

```
speeds = pypilot.Int32Vector([1000, 1000, 1000, 500, 500, 500])
sdk.armSpeedToServo(1, speeds)
```

2.7.45 暂停/恢复机械臂 (armPauseRobot)

```
def armPauseRobot(robotId: int, pause: bool) -> int:
```

参数说明:

- **robotId: int** - 机械臂 ID (1: 左臂, 2: 右臂, 3: 腰)
- **pause: bool** - True 表示暂停, False 表示恢复

返回值:

- **int** - 结果码, 0 表示成功, 非 0 表示失败

使用示例:

```
sdk.armPauseRobot(1, True)    # 暂停机械臂
sdk.armPauseRobot(1, False)   # 恢复机械臂
```

2.7.46 清除机械臂路径规划 (armClearRobotRoute)

```
def armClearRobotRoute(robotId: int, emergency_stop: bool = True) ->
int:
```

参数说明:

- **robotId: int** - 机械臂 ID (1: 左臂, 2: 右臂, 3: 腰) , -1 表示所有机械臂
- **emergency_stop: bool** - 是否立即停止, 默认 True。为 False 时机械臂会完成当前规划的路径再停止。

返回值:

- **int** - 结果码, 0 表示成功, 非 0 表示失败

使用示例:

```
sdk.armClearRobotRoute(1, True) # 立即停止机械臂 1
sdk.armClearRobotRoute(-1)     # 停止所有机械臂
```

2.7.47 获取机械臂暂停状态 (armGetRobotPaused)

```
def armGetRobotPaused(robotId: int) -> tuple[int, bool]:
```

参数说明:

- **robotId: int** - 机械臂 ID (1: 左臂, 2: 右臂, 3: 腰)

返回值:

- **tuple[int, bool]** - (接口调用结果 (0 成功, 非 0 失败), 暂停状态 (True 表示已暂停且停稳, False 表示运动中或减速中))

使用示例:

```
result, pause= sdk.armGetRobotPaused(1)
if pause:
    print("机械臂 1 已暂停并停稳")
```

2.7.48 设置到位容差 (armSetEnableTolerance)

```
def armSetEnableTolerance(robot_id: int, enable: bool, tolerance: float
= 0.1, tolerance_time: float = 0.5) -> int:
```

参数说明:

- **robotId: int** - 机械臂 ID (1: 左臂, 2: 右臂, 3: 腰)
- **enable: bool** - True 开启到位容差, False 关闭

- **tolerance: float** - 到位容差 (角度, 最小 0.1)
- **tolerance_time: float** - 到位容差时间 (秒, 最小 0.5)

返回值:

- **int** - 是否设置成功 (0 成功, 非 0 失败)

使用示例:

```
sdk.armSetEnableTolerance(1, True, 0.5, 1.0)
```

2.7.49 获取机械臂运动状态 (armGetRobotMoveState)

```
def armGetRobotMoveState(robot_id: int) -> tuple[int, bool]:
```

参数说明:

- **robotId: int** - 机械臂 ID (1: 左臂, 2: 右臂, 3: 腰)

返回值:

- **tuple[int, bool]** - (接口调用结果 (0 成功, 非 0 失败), 运动状态)

使用示例:

```
result, state = sdk.armGetRobotMoveState(1)
if state:
    print("机械臂运动中")
elif:
    print("机械臂已停止")
```

2.7.50 获取轴运动状态 (armGetRobotAxisMoveState)

```
def armGetRobotAxisMoveState(robot_id: int, axis_id: int) -> tuple[int, bool]:
```

参数说明:

- **robotId: int** - 机械臂 ID (1: 左臂, 2: 右臂, 3: 腰)
- **axis_id: int** - 轴索引 (从 0 开始)

返回值:

- **tuple[int, bool]** - (接口调用结果 (0 成功, 非 0 失败), 运动状态)

使用示例:

```
state = sdk.armGetRobotAxisMoveState(1, 0)
if state == 1:
    print("轴 0 运动中")
```

2.7.51 获取轴极限值 (armGetAxisParameter)

```
def armGetAxisParameter(robot_id: int, axis_id: int, index: int) -> tuple[int, float]:
```

参数说明:

- **robotId: int** - 机械臂 ID (1: 左臂, 2: 右臂, 3: 腰)
- **axis_id: int** - 轴索引 (从 0 开始)

- **index: int** - 轴参数索引 (1: 正极限, 2: 负极限)

返回值:

- **tuple[int, float]** - (接口调用结果 (0 成功, 非 0 失败), 参数值)

使用示例:

```
success, value = sdk.armGetAxisParameter(1, 0, 1)
if success:
    print(f"轴 0 正极限: {value}")
```

2.7.52 设置轴极限值 (armSetAxisParameter)

```
def armSetAxisParameter(robot_id: int, axis_id: int, index: int,
                        limitation: float) -> int:
```

参数说明:

- **robotId: int** - 机械臂 ID (1: 左臂, 2: 右臂, 3: 腰)
- **axis_id: int** - 轴索引 (从 0 开始)
- **index: int** - 轴参数索引 (1: 正极限, 2: 负极限)
- **limitation: float** - 极限值

返回值:

- **int** - 是否设置成功 (0 成功, 非 0 失败)

使用示例:

```
sdk.armSetAxisParameter(1, 0, 1, 180.0) # 设置轴 0 正极限为 180 度
```

2.7.53 世界坐标转关节坐标 (armTryWorlds)

```
def armTryWorlds(robotId: int, worlds: DoubleVector) -> tuple[int, DoubleVector]:
```

参数说明:

- **robotId: int** - 机械臂 ID (1: 左臂, 2: 右臂, 3: 腰)
- **worlds: DoubleVector** - 目标点在世界坐标系中的坐标 [X, Y, Z, U, V, W]

返回值:

- **tuple[int, DoubleVector]** - (接口调用结果 (0 成功, 非 0 失败), 关节坐标)

使用示例:

```
worlds = pypilot.DoubleVector([100, 200, 300, 0, 0, 0])
result, joints = sdk.armTryWorlds(1, worlds)
```

2.7.54 检查关节是否超限 (armJointOutLimit)

```
def armJointOutLimit(robotId: int, axis_id: int, joint: float) -> tuple[int, bool, bool]:
```

参数说明:

- **robotId: int** - 机械臂 ID (1: 左臂, 2: 右臂, 3: 腰)
- **axis_id: int** - 轴索引 (从 0 开始)
- **joint: float** - 目标关节角度

返回值:

- **tuple[int, bool, bool]** - (接口调用结果 (0 成功, 非 0 失败), 是否超正极限, 是否超负极限)

使用示例:

```
result, over_pos, over_neg = sdk.armJointOutLimit(1, 0, 200.0)
if over_pos:
    print("轴 0 超过正极限")
```

2.7.55 开启/关闭拖动示教 (armActivateDragTeach)

```
def armActivateDragTeach(robot_id: int, sw: bool, friction: list[float]
= None, compensate: list[float]= None) -> int:
```

参数说明:

- **robotId: int** - 机械臂 ID (1: 左臂, 2: 右臂, 3: 腰)
- **sw: bool** - True 开启拖动示教, False 关闭
- **friction: list[float]**- 每个轴的摩擦比例, 可选参数
- **compensate: list[float]**- 每个轴的补偿系数, 可选参数

返回值:

- **int** -是否设置成功 (0 成功, 非 0 失败)

使用示例:

```
sdk.armActivateDragTeach(1, True) # 开启拖动示教
sdk.armActivateDragTeach(1, False) # 关闭拖动示教
```

2.7.56 开启/关闭碰撞检测 (armActivateCollision)

```
def armActivateCollision(robot_id: int, sw: bool, level: int) -> int:
```

参数说明:

- **robotId: int** - 机械臂 ID (1: 左臂, 2: 右臂, 3: 腰)
- **sw: bool** - True 开启碰撞检测, False 关闭
- **level: int** - 碰撞检测等级, 范围[1, 10], 等级越小越灵敏

返回值:

- **int** - 是否设置成功 (0 成功, 非 0 失败)

使用示例:

```
sdk.armActivateCollision(1, True, 1)
```

2.7.57 恒力模式切换 (armForceKeepSwitch)

```
def armForceKeepSwitch(robotId: int, state: int, type: int = -1, fm: float = -1, startPos: list[float]= None, adm_param: list[float]= None) -> int:
```

参数说明:

- **robotId: int** - 机械臂 ID (1: 左臂, 2: 右臂, 3: 腰)

- **state: int** – 0 表示退出恒力模式, 1 表示进入恒力模式
- **type: int** – 恒力类型, 0 表示恒力跟踪, 1 表示装配
- **fm: float** – 恒力期望值 (合力)
- **startPos: list[float]**– 起始末端位姿 (世界坐标系)
- **adm_param: list[float]**– 力控参数 (M, B, K, P, I, D)

返回值:

- **int** – 结果码, 0 表示成功, 非 0 表示失败

使用示例:

```
sdk.armForceKeepSwitch(1, 1, 0, 10.0, [100, 200, 300, 0, 0, 0], [1, 0, 100, 1, 0, 0])
```

2.7.58 获取力控状态 (armGetForceKeepState)

```
def armGetForceKeepState(robotId: int) -> tuple[int, int]:
```

参数说明:

- **robotId: int** – 机械臂 ID (1: 左臂, 2: 右臂, 3: 腰)

返回值:

- **tuple[int, int]** – (接口调用结果 (0 成功, 非 0 失败), 力控状态)

力控状态如下:

- 1: 寻面中
- 2: 寻面轨迹运行完成

- 3: 恒力跟踪
- 4: 暂停
- 5: 停止

使用示例:

```
state = sdk.armGetForceKeepState(1)
if state == 3:
    print("恒力跟踪中")
```

2.7.59 导纳控制初始化 (armAdmittanceControlInit)

```
def armAdmittanceControlInit(robotId: int, adm_param_6dof:
    IntDoubleVectorMap]) -> int:
```

参数说明:

- **robotId: int** - 机械臂 ID (1: 左臂, 2: 右臂, 3: 腰)
- **adm_param_6dof: IntDoubleVectorMap** - 六维力控参数 {维度: [M, B, K, P, I, D]}

返回值:

- **int** - 结果码, 0 表示成功, 非 0 表示失败

使用示例:

```
params = pypilot.IntDoubleVectorMap()
params[0] = pypilot.DoubleVector([1.0, 0.5, 100.0, 1.0, 0.0, 0.0])
params[1] = pypilot.DoubleVector([1.0, 0.5, 100.0, 1.0, 0.0, 0.0])
sdk.armAdmittanceControlInit(1, params)
```

2.7.60 导纳控制 (armAdmittanceControl)

```
def armAdmittanceControl(robotId: int, forces: DoubleVector) -> int:
```

参数说明:

- **robotId: int** - 机械臂 ID (1: 左臂, 2: 右臂, 3: 腰)
- **forces: DoubleVector** - 目标六维力

返回值:

- **int** - 结果码, 0 表示成功, 非 0 表示失败

使用示例:

```
forces = pypilot.DoubleVector([0.0, 0.0, 10.0, 0.0, 0.0, 0.0])
sdk.armAdmittanceControl(1, forces)
```

2.7.61 获取当前绑定模式 (armGetArmBindMode)

```
def armGetArmBindMode() -> tuple[int, int]:
```

返回值:

- **tuple[int, int]** - (结果码 (0 成功, 非 0 失败), 绑定模式)

绑定模式取值范围如下:

- 0: 无绑定。

- 1: 左臂绑定腰部。
- 2: 右臂绑定腰部
- 3: 双臂绑定腰部

各绑定模式的含义参见下文 `armBindModeWorldMove` 接口。

使用示例:

```
success, mode = sdk.armGetArmBindMode()
if success:
    print(f"绑定模式: {mode}")
```

2.7.62 绑定模式下世界坐标运动 (armBindModeWorldMove)

```
def armBindModeWorldMove(mode: int, robotWorlds: RobotWorldsDataVector)
-> int:
```

参数说明:

- **mode: int** - 绑定模式
 - 0: 无绑定。各机械臂需要分别传入目标坐标进行独立运动
 - 1: 左臂绑定腰部。单绑模式下, 指定手臂与腰部进行绑定, 仅需给出指定手臂的目标坐标。机器人会提前解算腰部不动时手臂能否抵达指定坐标, 如手臂到不了, 会再叠加腰部运动进行判断, 如能抵达, 手腰会同时运动 (同时开始但不是同时结束)
 - 2: 右臂绑定腰部。参见上方单绑模式说明
 - 3: 双臂绑定腰部。双绑模式下, 两个手臂与腰部进行绑定, 需给出两个手臂的目标坐标。机器人会提前解算腰部不动时手臂能否抵达指定坐标, 如手

臂到不了，会再叠加腰部运动进行判断，如能抵达，手腰会同时运动（同时开始但不是同时结束）

- **注意：**切换绑定模式需要等上个模式运动结束后再切换，否则返回失败。同种模式下允许连续调用运动函数
- **robotWorlds: RobotWorldsDataVector** – 指定机械臂及坐标[{"robotId": int, "worldPos": [X,Y,Z,U,V,W], "needCallback": bool}]
 - robotId: 机械臂 ID (1: 左臂, 2: 右臂, 3: 腰)
 - worldPos: 目标点在世界坐标系中的坐标 [X, Y, Z, U, V, W]
 - needCallback: 是否调用回调。回调函数用于通知运动是否结束，详见第 3 章 回调函数

返回值：

- **int** – 接口调用结果 (0 成功, 非 0 失败)

使用示例：

```
# 单绑模式
robot_worlds = [{"robotId": 1, "worldPos": [100, 200, 300, 0, 0, 0],
"needCallback": False}]
sdk.armBindModeWorldMove(1, robot_worlds)

# 双绑模式
robot_worlds = [
    {"robotId": 1, "worldPos": [100, 200, 300, 0, 0, 0],
"needCallback": False},
    {"robotId": 2, "worldPos": [150, 250, 350, 0, 0, 0],
"needCallback": False}
]
sdk.armBindModeWorldMove(3, robot_worlds)
```

2.7.63 机械臂加载负载 (armSetRobotLoad)

```
def armSetRobotLoad(self, robot_id: int, parameter: LoadParameter) -> int:
```

参数说明:

需要在机械臂末端实际增加了负载（如抓取了物品）后调用。

- **robotId: int** - 机械臂 ID (1: 左臂, 2: 右臂, 3: 腰)
- **parameter: LoadParameter** - 负载参数, 详见第 4 章 数据类型

返回值:

- **int** - 结果码, 0 表示成功, 非 0 表示失败

使用示例:

```
load_p = LoadParameter()
load_p.quality = 1.1
load_p.center_mass_x = 0.0
load_p.center_mass_y = 0.0
load_p.center_mass_z = 0.0
result = sdk.armSetRobotLoad(1, load_p)
if result == 0:
    print(f"机械臂 1 加载负载成功")
```

2.7.64 机械臂卸载负载 (armUnsetLoad)

```
def armUnsetLoad(self, robot_id: int) -> int:
```

参数说明:

需要在机械臂末端实际减少了负载（如放置了物品）后调用。

- **robotId: int** - 机械臂 ID (1: 左臂, 2: 右臂, 3: 腰)

返回值:

- **int** - 结果码, 0 表示成功, 非 0 表示失败

使用示例:

```
result = sdk.armUnsetLoad(1)
if result == 0:
    print(f"机械臂 1 (左臂) 卸载负载成功")
```

2.7.65 设置位置刷新周期 (armSetPosReFreshMS)

```
def armSetPosReFreshMS(self, robotId: int, jointMs: int, worldMs: int)
-> int:
```

参数说明:

- **robotId: int** - 机械臂 ID (1: 左臂, 2: 右臂, 3: 腰)
- **jointMs: int** - 关节位置反馈值刷新周期, 单位毫秒 (默认 40ms)
- **worldMs: int** - 世界位置反馈值刷新周期, 单位毫秒 (默认 50ms)

返回值:

- **int** - 结果码, 0 表示成功, 非 0 表示失败

使用示例:

```
result = sdk.armSetPosReFreshMS(1,40,40)
```

```
if result == 0:
    print(f"机械臂 1 设置位置刷新周期成功")
```

2.7.66 获取位置刷新周期 (armGetPosReFreshMS)

```
def armGetPosReFreshMS(self, robotId: int) -> tuple[int, int, int]:
```

参数说明:

- **robotId: int** - 机械臂 ID (1: 左臂, 2: 右臂, 3: 腰)

返回值:

- **tuple[int, int, int]** - [结果码 (0 成功, 非 0 失败), 关节位置刷新周期 (ms), 世界位置刷新周期 (ms)]

使用示例:

```
result, jointMs, worldMs = sdk.armGetPosReFreshMS(1)
if result == 0:
    print(f"机械臂 1 关节坐标刷新周期为{jointMs}ms, 世界坐标刷新周期为{worldMs}ms")
```

2.7.67 获取机械臂软急停状态

(armGetRobotSoftEmergencyStopStatus)

```
def armGetRobotSoftEmergencyStopStatus() -> tuple[int, bool]
```

返回值:

- **tuple[int, bool]** - [结果码 (0 成功, 非 0 失败), 软急停状态]

所有机械臂共用软急停状态:

- True 表示非急停状态
- False 表示所有机械臂急停中

使用示例:

```
armGetRobotSoftEmergencyStopStatus
result, status = sdk.armGetRobotSoftEmergencyStopStatus()
if result == 0 and not status:
    print(f"当前处于软急停状态")
```

2.7.68 机械臂软急停控制 (armRobotSoftEmergencyStop)

```
def armRobotSoftEmergencyStop(notstop: bool) -> int
```

参数说明:

- **notStop: bool** - 软急停触发标志, 所有机械臂共用同一个标志。True 表示解除软急停, False 表示触发软急停

返回值:

- **int** - 结果码, 0 表示成功, 非 0 表示失败

使用示例:

```
result= sdk.armRobotSoftEmergencyStop(False)
if result == 0:
```

```
print(f"触发软急停成功")
```

2.8 系统管理 & 监控

2.8.1 RCS 连接管理 (rcsConnectionManage)

```
def rcsConnectionManage(self, cmd: str) -> int
```

参数说明:

- **cmd: str** - 管理命令字符串
 - **ONLINE**: 连接 RCS
 - **OFFLINE**: 断开连接

返回值:

- **int** - 结果码, 0 表示成功, 非 0 表示失败

使用示例:

```
result = sdk.rcsConnectionManage("ONLINE")
if result == 0:
    print("RCS 连接管理命令执行成功")
```

2.9 其他

2.9.1 获取扩展参数列表 (getExtendedParameters)

```
def getExtendedParameters(self) -> Tuple[int, List[Parameter]]:
```

返回值:

- **Tuple[int, List[Parameter]]** - (结果码 (0 表示成功, 非 0 表示失败), 扩展参数列表)

使用示例:

```
result_code, params = sdk.getExtendedParameters()
if result_code == 0:
    print(f"扩展参数数量: {len(params)}")
    for param in params:
        print(f"参数: {param.key} = {param.value}")
```

第3章 回调函数

3.1 回调函数类型定义

```
ImuCallback = Callable[['ImuData'], None]
OdomCallback = Callable[['OdomData'], None]
VelocityCallback = Callable[['VelocityData'], None]
CurrentPoseCallback = Callable[['CurrentPoseData'], None]
QrDataCallback = Callable[['QrData'], None]
CameraCallback = Callable[['CameraData'], None]
LidarCallback = Callable[['LidarData'], None]
MoveJointsEndCallback = Callable[['MoveJointsEndData'], None]
MoveWorldsEndCallback = Callable[['MoveWorldsEndData'], None]
```



NOTE

回调数据的数据结构请参见第 4 章 数据类型。

3.2 设置回调函数

3.2.1 设置 IMU 数据回调 (setImuCallback)

```
def setImuCallback(self, py_func: ImuCallback) -> int:
```

参数说明:

- **py_func: ImuCallback** - 接收 ImuData 的回调函数

返回值:

- **int** - 结果码, 0 表示成功, 非 0 表示失败

使用示例:

```
def imu_callback(imu_data):
    print(f"IMU 数据: 角速度({imu_data.angular_vel_x:.2f},
{imu_data.angular_vel_y:.2f}, {imu_data.angular_vel_z:.2f})")
    print(f"线性加速度: ({imu_data.linear_acc_x:.2f},
{imu_data.linear_acc_y:.2f}, {imu_data.linear_acc_z:.2f})")

# 设置回调
if sdk.setImuCallback(imu_callback) == 0:
    print("IMU 回调设置成功")
else:
    print("IMU 回调设置失败")
```

3.2.2 设置里程计数据回调 (setOdomCallback)

```
def setOdomCallback(self, py_func: OdomCallback) -> int:
```

参数说明:

- **py_func: OdomCallback** - 接收 OdomData 的回调函数

返回值:

- **int** - 结果码, 0 表示成功, 非 0 表示失败

使用示例:

```
def odom_callback(odom_data):
    print(f"位置: ({odom_data.pos_x:.2f}, {odom_data.pos_y:.2f},
{odom_data.pos_z:.2f})")
    print(f"线速度: ({odom_data.linear_x:.2f}, {odom_data.linear_y:.2f},
{odom_data.linear_z:.2f})")
```

```
if sdk.setOdomCallback(odom_callback) == 0:  
    print("里程计回调设置成功")
```

3.2.3 设置速度数据回调 (setVelocityCallback)

```
def setVelocityCallback(self, py_func: VelocityCallback) -> int:
```

参数说明:

- **py_func: VelocityCallback** - 接收 VelocityData 的回调函数

返回值:

- **int** - 结果码, 0 表示成功, 非 0 表示失败

使用示例:

```
def velocity_callback(velocity_data):  
    print(f"实际速度: vx={velocity_data.act_vel_x:.2f},  
vy={velocity_data.act_vel_y:.2f}, omega={velocity_data.act_vel_w:.2f}")  
  
if sdk.setVelocityCallback(velocity_callback) == 0:  
    print("速度回调设置成功")
```

3.2.4 设置当前位姿回调 (setCurrentPoseCallback)

```
def setCurrentPoseCallback(self, py_func: CurrentPoseCallback) -> int:
```

参数说明:

- **py_func: CurrentPoseCallback** - 接收 CurrentPoseData 的回调函数

返回值:

- **int** - 结果码, 0 表示成功, 非 0 表示失败

使用示例:

```
def pose_callback(pose_data):
    print(f"当前位姿: ({pose_data.pos_x:.2f}, {pose_data.pos_y:.2f}, {math.degrees(pose_data.pos_angle):.2f}°)")
    print(f"当前站点: {pose_data.pos_current_station}")

if sdk.setCurrentPoseCallback(pose_callback) == 0:
    print("当前位姿回调设置成功")
```

3.2.5 设置二维码数据回调 (setQrDataCallback)

```
def setQrDataCallback(self, qr_id: int, py_func: QrDataCallback) -> int:
```

参数说明:

隙锋当前版本未安装二维码相机, 无需使用该回调。

- **qr_id: int** - 二维码相机 ID
- **py_func: QrDataCallback** - 接收 QrData 的回调函数

返回值:

- **int** - 结果码, 0 表示成功, 非 0 表示失败

使用示例:

```
def qr_callback(qr_data):
```

```
print(f"检测到二维码: 签名数量={len(qr_data.signatures)}")
print(f"位置: ({qr_data.x:.2f}, {qr_data.y:.2f}), 角度
={math.degrees(qr_data.theta):.2f}°")

if sdk.setQrDataCallback(1, qr_callback) == 0:
    print("二维码数据回调设置成功")
```

3.2.6 设置相机数据回调 (setCameraCallback)

```
def setCameraCallback(self, camera_id: int, py_func: CameraCallback) ->
int:
```

参数说明:

- **camera: int** - 相机 ID, 可通过 Toolkit 查询和配置
- **py_func: CameraCallback** - 接收 CameraData 的回调函数

返回值:

- **int** - 结果码, 0 表示成功, 非 0 表示失败

使用示例:

```
def camera_callback(camera_data):
    print(f"相机{camera_id}数据: 点云数量={len(camera_data.points)}")
    for i, point in enumerate(camera_data.points[:3]): # 显示前3个点
        print(f" 点{i+1}: ({point.x:.2f}, {point.y:.2f},
{point.z:.2f})")

if sdk.setCameraCallback(1, camera_callback) == 0:
    print("相机数据回调设置成功")
```

3.2.7 设置激光雷达数据回调 (setLidarDataCallback)

```
def setLidarDataCallback(self, lidar_id: int, py_func: LidarCallback)
-> int:
```

参数说明:

- **lidar_id: int** – 激光雷达 ID, 可通过 Toolkit 查询和配置
- **py_func: LidarCallback** – 接收 LidarData 的回调函数

返回值:

- **int** – 结果码, 0 表示成功, 非 0 表示失败

使用示例:

```
def lidar_callback(lidar_data):
    print(f"激光雷达{lidar_data.lidar_id}数据: 扫描点数
    ={lidar_data.range_count}")
    print(f"角度范围: {math.degrees(lidar_data.angle_min):.2f}° 到
    {math.degrees(lidar_data.angle_max):.2f}°")

if sdk.setLidarDataCallback(1, lidar_callback) == 0:
    print("激光雷达回调设置成功")
```

3.2.8 设置关节运动结束回调

(armSetMoveJointsEndCallback)

```
def armSetMoveJointsEndCallback(self, py_func: Callable[[int, bool],
None]) -> int:
```

参数说明:

- **py_func: Callable[[int, bool], None]** - 回调函数, 接收参数 (robot_id: int, end: bool)
 - **robot_id: int** - 机械臂 ID (1: 左臂, 2: 右臂, 3: 腰)
 - **end: bool** - 运动是否结束

返回值:

- **int** - 结果码, 0 表示成功, 非 0 表示失败

使用示例:

```
def on_joints_move_end(robot_id: int, end: bool):  
    if end:  
        print(f"机械臂{robot_id}关节运动已结束")  
    else:  
        print(f"机械臂{robot_id}关节运动未结束")  
  
if sdk.armSetMoveJointsEndCallback(on_joints_move_end) == 0:  
    print("关节运动回调设置成功")
```

3.2.9 设置世界运动结束回调

(armSetMoveWorldsEndCallback)

```
def armSetMoveWorldsEndCallback(self, py_func: Callable[[int, bool],  
None]) -> int:
```

参数说明:

- **py_func: Callable[[int, bool], None]** - 回调函数, 接收参数 (robot_id: int, end: bool)

- **robot_id: int** – 机械臂 ID (1: 左臂, 2: 右臂, 3: 腰)
- **end: bool** – 运动是否结束

返回值:

- **int** – 结果码, 0 表示成功, 非 0 表示失败

使用示例:

```
def on_worlds_move_end(robot_id: int, end: bool):  
    if end:  
        print(f"机械臂{robot_id}世界坐标运动已结束")  
        # 可以在这里执行下一步操作  
    else:  
        print(f"机械臂{robot_id}世界坐标运动未结束")  
  
if sdk.armSetMoveWorldsEndCallback(on_worlds_move_end) == 0:  
    print("世界运动回调设置成功")
```

3.3 取消回调函数

3.3.1 通用取消回调 (cancelCallback)

```
def cancelCallback (self, data_type: str) -> int:
```

参数说明:

- **data_type: str** – 需要取消的回调类型, 取值范围如下:
 - **imu_data**: IMU 回调
 - **odom_data**: 里程计回调
 - **vel_data**: 速度回调

- `cur_pose_data`: 当前位姿回调
- `qr_cam_<N>`: 二维码数据回调, <N>替换为设备 ID
- `camera_link_<N>`: 相机数据回调, <N>替换为设备 ID
- `lidar_data_<N>`: 激光雷达数据回调, <N>替换为设备 ID

返回值:

- **int** – 结果码, 0 表示成功, 非 0 表示失败

使用示例:

```
if sdk.cancelCallback ("lidar_data_1") == 0:  
    print ("雷达 1 回调已取消")
```

3.3.2 取消 IMU 回调 (`cancelImuCallback`)

```
def cancelImuCallback(self) -> int:
```

返回值:

- **int** – 结果码, 0 表示成功, 非 0 表示失败

使用示例:

```
if sdk.cancelImuCallback() == 0:  
    print("IMU 回调已取消")
```

3.3.3 取消里程计回调 (cancelOdomCallback)

```
def cancelOdomCallback(self) -> int:
```

返回值:

- **int** – 结果码, 0 表示成功, 非 0 表示失败

使用示例:

```
if sdk.cancelOdomCallback() == 0:  
    print("里程计回调已取消")
```

3.3.4 取消速度回调 (cancelVelocityCallback)

```
def cancelVelocityCallback(self) -> int:
```

返回值:

- **int** – 结果码, 0 表示成功, 非 0 表示失败

使用示例:

```
if sdk.cancelVelocityCallback() == 0:  
    print("速度回调已取消")
```

3.3.5 取消当前位姿回调 (cancelCurrentPoseCallback)

```
def cancelCurrentPoseCallback(self) -> int:
```

返回值:

- **int** – 结果码, 0 表示成功, 非 0 表示失败

使用示例:

```
if sdk.cancelCurrentPoseCallback() == 0:  
    print("当前位姿回调已取消")
```

3.3.6 取消二维码数据回调 (cancelQrDataCallback)

```
def cancelQrDataCallback(self, qr_id: int) -> int:
```

参数说明:

隙锋当前版本未安装二维码相机, 无需使用该回调。

- **qr_id: int** – 二维码相机 ID

返回值:

- **int** – 结果码, 0 表示成功, 非 0 表示失败

使用示例:

```
if sdk.cancelQrDataCallback(1) == 0:  
    print("二维码数据回调已取消")
```

3.3.7 取消相机数据回调 (cancelCameraCallback)

```
def cancelCameraCallback(self, camera_id: int) -> int:
```

参数说明:

- **camera: int** – 相机 ID, 可通过 Toolkit 查询和配置

返回值:

- **int** – 结果码, 0 表示成功, 非 0 表示失败

使用示例:

```
if sdk.cancelCameraCallback(1) == 0:  
    print("相机数据回调已取消")
```

3.3.8 取消激光雷达数据回调 (cancelLidarDataCallback)

```
def cancelLidarDataCallback(self, lidar_id: int) -> int:
```

参数说明:

- **lidar_id: int** – 激光雷达 ID, 可通过 Toolkit 查询和配置

返回值:

- **int** – 结果码, 0 表示成功, 非 0 表示失败

使用示例:

```
if sdk.cancelLidarDataCallback(1) == 0:  
    print("激光雷达数据回调已取消")
```

3.3.9 取消关节运动结束回调 (armCancelMoveJointsEndCallback)

```
def armCancelMoveJointsEndCallback(self) -> int:
```

返回值:

- **int** – 结果码, 0 表示成功, 非 0 表示失败

使用示例:

```
if sdk.armCancelMoveJointsEndCallback() == 0:  
    print("关节运动回调已取消")  
else:  
    print("取消关节运动回调失败")
```

3.3.10 取消世界运动结束回调 (armCancelMoveWorldsEndCallback)

```
def armCancelMoveWorldsEndCallback(self) -> int:
```

返回值:

- **int** – 结果码, 0 表示成功, 非 0 表示失败

使用示例:

```
if sdk.armCancelMoveWorldsEndCallback() == 0:  
    print("世界运动回调已取消")  
else:  
    print("取消世界运动回调失败")
```

第4章 数据类型

4.1 数据类型列表

类别	数据类型
基础数据	● 位置信息 ● 速度信息 ● 地图信息
硬件状态数据	● 动作状态 ● 电池信息 ● 错误信息 ● 按钮状态 ● 扩展参数 ● 驱动轮状态
查询和系统信息数据	● 二维码信息 ● AGV 基本信息 ● 计算机信息
几何数据	● 二维点 ● 三维点 ● 二维激光数据
传感器数据	● IMU 数据 ● 里程计数据 ● 激光雷达数据 ● 速度数据 ● 当前位姿数据 ● 二维码数据 ● 相机数据
机械臂相关数据	枚举类型 ● 轴参数索引

类别	数据类型
	● 手臂绑定模式 数据结构 ● 负载参数 ● 机械臂世界坐标数据
Vector 类型说明	● 浮点数向量 ● 双精度浮点数向量 ● 整数向量 ● 字符串向量 ● 布尔向量 ● 字符串映射

4.2 基础数据

4.2.1 位置信息 (Position)

```
class Position:
    status: str          # 定位状态 (NOT_LOCATED/LOCATED)
    localizationScore: int # 定位质量 (0-3)
    x: float             # X 坐标
    y: float             # Y 坐标
    theta: float         # 角度 (弧度)
    located: bool        # 是否已定位
```

字段说明:

- **status: str** - 定位状态, "NOT_LOCATED"表示未定位, "LOCATED"表示已定位
- **localizationScore: int** - 定位质量评分, 范围 0-3, 数值越高定位质量越好
- **x: float** - X 坐标值, 单位为米

- **y: float** - Y 坐标值, 单位为米
- **theta: float** - 角度值, 单位为弧度
- **located: bool** - 布尔值, True 表示已定位, False 表示未定位

4.2.2 速度信息 (Velocity)

```
class Velocity:
    vx: float      # X 方向速度 (米/秒)
    vy: float      # Y 方向速度 (米/秒)
    omega: float   # 绕 Z 轴转动速度 (弧度/秒)
```

字段说明:

- **vx: float** - X 轴方向线速度, 正值表示前进, 负值表示后退, 单位为米/秒
- **vy: float** - Y 轴方向线速度, 正值表示左移, 负值表示右移, 单位为米/秒
- **omega: float** - 绕 Z 轴角速度, 正值表示逆时针旋转, 负值表示顺时针旋转, 单位为弧度/秒

4.2.3 地图信息 (MapInfo)

```
class MapInfo:
    mapId: str      # 地图编号
    locationMapId: str # 定位图编号
    mapVersion: str # 地图版本
    mapDescription: str # 地图描述
```

字段说明:

- **mapId: str** - 当前使用的地图 (含路网) 唯一标识符
- **locationMapId: str** - 用于定位的地图 (轮廓图) 标识符
- **mapVersion: str** - 地图版本号字符串
- **mapDescription: str** - 地图的描述信息

4.3 硬件状态数据

4.3.1 动作状态 (ActionState)

```
class ActionState:
    """Action state structure."""
    actionId: str          # 动作唯一标识符
    actionType: str         # 动作类型
    actionDescription: str  # 动作描述
    actionStatus: str       # 动作状态
    (WAITING/INITIALIZING/RUNNING/PAUSED/FINISHED/FAILED/CANCELLED)
    actionResult: Dict[str, Any] # 动作执行结果
    actionErrorMsg: str       # 失败时的原因描述
```

字段说明:

- **actionId: str** - 动作的唯一标识符
- **actionType: str** - 动作类型, 如"MOVE"、"LIFT"、"ROTATE"等
- **actionDescription: str** - 动作的详细描述信息
- **actionStatus: str** - 动作状态
 - WAITING: 等待执行
 - INITIALIZING: 初始化中

- RUNNING: 正在执行
 - PAUSED: 已暂停
 - FINISHED: 执行完成
 - FAILED: 执行失败
 - CANCELLED: 已取消
- **actionResult: Dict[str, Any]** - 动作执行结果, 包含具体的输出数据
 - **actionErrorMsg: str** - 执行失败时的原因描述

4.3.2 电池信息 (BatteryInfo)

```
class BatteryInfo:
    batteryId: str          # 电池 ID, 电池序列号
    batteryLevel: float     # 当前电量 (%)
    batteryVoltage: float   # 电池电压 (V)
    batteryCharge: float    # 电池充电电流 (A)
    batteryDischarge: float # 电池放电电流 (A)
    chargingTimes: int      # 电池充放电次数
    batteryHealth: int      # 电池健康状态 (0-100)
    batteryCellTemperatures: IntVector # 电池电芯温度
    batteryMosfetTemperatures: IntVector # 电池mosfet 温度
    vcells: IntVector       # 电池电芯电压
    charging: bool          # 是否正在充电
    batteryFcc: int         # 满电容量 (AH)
    batteryRc: int          # 剩余容量 (AH)
    softwareVersionNumber: str # 电池软件版本号
    hardwareVersionNumber: str # 电池硬件版本号
    warningValue: int       # 警告状态异常码
    errorValue: int         # 错误状态异常码
    protectValue: int       # 保护状态异常码
    batteryMaxCellTemperature: int # 电芯最高温度 (°C)
    batteryMinCellTemperature: int # 电芯最低温度 (°C)
    batteryMaxCellVoltages: int  # 电芯最高电压 (mV)
    batteryMinCellVoltages: int  # 电芯最低电压 (mV)
```

```
batteryCellVlotageGap: int      # 电芯电压差 (mV)
```

字段说明:

- **batteryId: str** - 电池的唯一序列号标识
- **batteryLevel: float** - 当前电量百分比, 范围 0-100
- **batteryVoltage: float** - 电池当前电压, 单位为伏特
- **batteryCharge: float** - 充电电流, 正数表示充电, 单位为安培
- **batteryDischarge: float** - 放电电流, 正数表示放电, 单位为安培
- **chargingTimes: int** - 电池完整充放电循环次数
- **batteryHealth: int** - 电池健康度评分, 范围 0-100, 数值越低健康度越差
- **batteryCellTemperatures: IntVector** - 各电芯温度数组, 单位为摄氏度
- **batteryMosfetTemperatures: IntVector** - MOSFET 温度数组, 单位为摄氏度
- **vcells: IntVector** - 各电芯电压数组, 单位为毫伏
- **charging: bool** - 充电状态标志, True 表示正在充电
- **batteryFcc: int** - 电池满电容量, 单位为安时
- **batteryRc: int** - 电池剩余容量, 单位为安时
- **softwareVersionNumber: str** - 电池软件版本号
- **hardwareVersionNumber: str** - 电池硬件版本号
- **warningValue: int** - 警告状态异常码
- **errorValue: int** - 错误状态异常码
- **protectValue: int** - 保护状态异常码
- **batteryMaxCellTemperature: int** - 电芯最高温度, 单位为摄氏度

- **batteryMinCellTemperature: int** - 电芯最低温度, 单位为摄氏度
- **batteryMaxCellVoltages: int** - 电芯最高电压, 单位为毫伏
- **batteryMinCellVoltages: int** - 电芯最低电压, 单位为毫伏
- **batteryCellVlotageGap: int** - 电芯最大电压差, 单位为毫伏

4.3.3 错误信息 (ErrorInfo)

```
class ErrorInfo:
    """Error information structure."""
    errorCode: str # 错误编码
    errorDescription: str # 详细描述
    errorHint: str # 解决提示
    errorLevel: str # 报警等级 (WARNING/FATAL)
```

字段说明:

- **errorCode: str** - 错误的唯一编码标识符
- **errorDescription: str** - 错误的详细描述信息
- **errorHint: str** - 错误的解决建议和提示
- **errorLevel: str** - 报警等级
 - WARNING: 警告级别, 不影响主要功能
 - FATAL: 致命错误, 需要立即处理

4.3.4 按钮状态 (ButtonStatus)

```
class ButtonStatus:
    """Button status structure."""
    powerButton: int # 底盘开关机按钮触发状态
```

```
resetButton: int # 复位按钮触发状态
```

字段说明:

- **powerButton: int** - 电源按钮状态, 0 表示未按下, 1 表示已按下
- **resetButton: int** - 复位按钮状态, 0 表示未按下, 1 表示已按下

4.3.5 扩展参数 (Parameter)

```
class Parameter:  
    """Extended parameter structure."""  
    key: str # 扩展参数名称  
    value: str # 与键对应的值
```

字段说明:

- **key: str** - 参数的名称标识符
- **value: str** - 参数对应的字符串值

4.3.6 驱动轮状态 (DrivingWheelInfo)

```
class DrivingWheelInfo:  
    drivingWheelId: str # 驱动轮id  
    status: str # TRIGGER/NOT_TRIGGER/CLOSE/ERROR
```

字段说明:

- **drivingWheelId: str** - 驱动轮 ID
- **status: str** - 驱动轮状态

- TRIGGER: 驱动中
- NOT_TRIGGER: 待机中
- CLOSE: 已关机
- ERROR: 报错状态

4.4 查询和系统信息数据

4.4.1 二维码信息 (QrInfo)

```
class QrInfo:
    """QR code information structure."""
    qrId: int      # 二维码 ID
    x: float       # X 坐标
    y: float       # Y 坐标
    theta: float   # 角度
```

字段说明:

隙锋当前版本未安装二维码相机，无需使用该数据接口。

- **qrId: int** - 二维码的唯一数字标识符
- **x: float** - 二维码在地图中的 X 坐标，单位为米
- **y: float** - 二维码在地图中的 Y 坐标，单位为米
- **theta: float** - 二维码的朝向角度，单位为弧度

4.4.2 AGV 基本信息 (AgvInfo)

```
class AgvInfo:
    """AGV basic information structure."""
    agvId: str      # AGV ID
```

```
seriesName: str          # 系列名称
serialNumber: str        # 序列号
versions: Dict[str, str] # 版本信息字典
```

字段说明:

- **agvId: str** - AGV 设备的唯一标识符
- **seriesName: str** - AGV 产品系列名称
- **serialNumber: str** - AGV 设备的序列号
- **versions: Dict[str, str]** - 版本信息字典, 包含各组件的版本号

4.4.3 计算机信息 (ComputerInfo)

```
class ComputerInfo:
    """Computer information structure."""
    cpu_ocp: float      # CPU 占用率
    mem_ocp: float      # 内存占用率
    space_free: float   # 剩余磁盘空间
    temperature: float  # 温度
    space_total: float  # 总磁盘空间
    mem_total: float    # 总内存
    hddisk_life: int    # 硬盘寿命
    CPU: str           # CPU 信息
    system: str         # 系统信息
```

字段说明:

- **cpu_ocp: float** - CPU 使用率, 百分比形式 (0-100)
- **mem_ocp: float** - 内存使用率, 百分比形式 (0-100)
- **space_free: float** - 剩余磁盘空间, 单位为 GB

- **temperature: float** - 系统温度，单位为摄氏度
- **space_total: float** - 磁盘总空间，单位为 GB
- **mem_total: float** - 内存总量，单位为 GB
- **harddisk_life: int** - 硬盘剩余寿命百分比 (0-100)
- **CPU: str** - CPU 型号和详细信息
- **system: str** - 操作系统信息

4.5 几何数据

4.5.1 二维点 (Point2D)

```
class Point2D:
    """2D point structure."""
    x: float # X 坐标
    y: float # Y 坐标
```

字段说明:

- **x: float** - 二维空间中的 X 坐标值
- **y: float** - 二维空间中的 Y 坐标值

4.5.2 三维点 (Point3D)

```
class Point3D:
    """3D point structure."""
    x: float # X 坐标
    y: float # Y 坐标
    z: float # Z 坐标
```

字段说明:

- **x: float** – 三维空间中的 X 坐标值
- **y: float** – 三维空间中的 Y 坐标值
- **z: float** – 三维空间中的 Z 坐标值

4.5.3 二维激光数据 (LaserData2D)

```
class LaserData2D:
    """2D laser scan data structure."""
    points: List[Point2D] # 点云数据
```

字段说明:

- **points: List[Point2D]** – 二维激光扫描得到的点云数据列表

4.6 传感器数据

4.6.1 IMU 数据 (ImuData)

```
class ImuData:
    seq: int # 序列号
    timestamp_sec: int # 时间戳秒
    timestamp_nsec: int # 时间戳纳秒
    frame_id: str # 坐标系 ID
    orientation_x: float # 四元数 x
    orientation_y: float # 四元数 y
    orientation_z: float # 四元数 z
    orientation_w: float # 四元数 w
    orientation_covariance: DoubleVector # 方向协方差矩阵 (9 elements)
    angular_vel_x: float # 角速度 x
    angular_vel_y: float # 角速度 y
    angular_vel_z: float # 角速度 z
```

```

    angular_velocity_covariance: DoubleVector # 角速度协方差矩阵 (9
elements)
    linear_acc_x: float # 线性加速度 x
    linear_acc_y: float # 线性加速度 y
    linear_acc_z: float # 线性加速度 z
    linear_acc_covariance: DoubleVector # 线性加速度协方差矩阵 (9
elements)

```

4.6.2 里程计数据 (OdomData)

```

class OdomData:
    seq: int # 序列号
    timestamp_sec: int # 时间戳秒
    timestamp_nsec: int # 时间戳纳秒
    frame_id: str # 坐标系 ID
    child_frame_id: str # 子坐标系 ID
    pos_x: float # x 位置
    pos_y: float # y 位置
    pos_z: float # z 位置
    ori_x: float # 四元数 x
    ori_y: float # 四元数 y
    ori_z: float # 四元数 z
    ori_w: float # 四元数 w
    linear_x: float # 线速度 x
    linear_y: float # 线速度 y
    linear_z: float # 线速度 z
    angular_x: float # 角速度 x
    angular_y: float # 角速度 y
    angular_z: float # 角速度 z
    pose_covariance: DoubleVector # 位置协方差矩阵 (36 elements)
    twist_covariance: DoubleVector # 扭曲协方差矩阵 (36 elements)

```

4.6.3 激光雷达数据 (LidarData)

```

class LidarData:

```

```
lidar_id: int          # 激光雷达 ID
seq: int               # 序列号
timestamp_sec: int      # 时间戳秒
timestamp_nsec: int     # 时间戳纳秒
frame_id: str          # 坐标系 ID
angle_min: float        # 最小角度
angle_max: float        # 最大角度
angle_increment: float  # 角度增量
time_increment: float   # 时间增量
scan_time: float        # 扫描时间
range_min: float        # 最小距离
range_max: float        # 最大距离
max_count: int          # 最大点数
range_count: int        # 实际距离点数
intensity_count: int    # 实际强度点数
ranges: FloatVector     # 距离数组
intensities: FloatVector # 强度数组
```

4.6.4 速度数据 (VelocityData)

```
class VelocityData:
    """Velocity data structure."""
    act_vel_x: float # 实际速度 x
    act_vel_y: float # 实际速度 y
    act_vel_w: float # 实际角速度 w
```

字段说明:

- **act_vel_x: float** - AGV 在 X 方向的实际线速度, 单位为米/秒
- **act_vel_y: float** - AGV 在 Y 方向的实际线速度, 单位为米/秒
- **act_vel_w: float** - AGV 绕 Z 轴的实际角速度, 单位为弧度/秒

4.6.5 当前位姿数据 (CurrentPoseData)

```
class CurrentPoseData:
    """Current pose data structure."""
    seq: int          # 序列号
    timestamp_sec: int # 时间戳秒
    timestamp_nsec: int # 时间戳纳秒
    frame_id: str      # 坐标系 ID
    pos_x: float       # x 位置
    pos_y: float       # y 位置
    pos_angle: float   # 角度
    pos_current_station: str # 当前站点
```

字段说明:

- **seq: int** - 数据包序列号, 用于标识数据包的顺序
- **timestamp_sec: int** - 时间戳的秒部分, Unix 时间戳
- **timestamp_nsec: int** - 时间戳的纳秒部分, 用于更精确的时间标识
- **frame_id: str** - 坐标系标识符, 说明数据所在的参考坐标系
- **pos_x: float** - AGV 在 X 方向的位置坐标, 单位为米
- **pos_y: float** - AGV 在 Y 方向的位置坐标, 单位为米
- **pos_angle: float** - AGV 的朝向角度, 单位为弧度
- **pos_current_station: str** - AGV 当前所在的站点名称或 ID

4.6.6 二维码数据 (QrData)

```
class QrData:
    """QR code data structure."""
    seq: int          # 序列号
    timestamp_sec: int # 时间戳秒
```

```
timestamp_nsec: int    # 时间戳纳秒
frame_id: str          # 坐标系 ID
signatures: StringVector # 签名列表
x: float               # x 坐标
y: float               # y 坐标
theta: float           # 角度
```

字段说明:

隙锋当前版本未安装二维码相机，无需使用该数据结构。

- **seq: int** - 数据包序列号，用于标识数据包的顺序
- **timestamp_sec: int** - 时间戳的秒部分
- **timestamp_nsec: int** - 时间戳的纳秒部分
- **frame_id: str** - 坐标系标识符，说明二维码坐标的参考坐标系
- **signatures: StringVector** - 识别到的二维码签名列表，包含检测到的所有二维码标识
- **x: float** - 二维码在地图中的 X 坐标，单位为米
- **y: float** - 二维码在地图中的 Y 坐标，单位为米
- **theta: float** - 二维码的朝向角度，单位为弧度

4.6.7 相机数据 (CameraData)

```
class CameraData:
    """Camera data structure."""
    seq: int          # 序列号
    timestamp_sec: int # 时间戳秒
    timestamp_nsec: int # 时间戳纳秒
    frame_id: str      # 坐标系 ID
    points: Point3DVector # 点云数据
```

字段说明:

- **seq: int** - 数据包序列号, 用于标识数据包的顺序
- **timestamp_sec: int** - 时间戳的秒部分
- **timestamp_nsec: int** - 时间戳的纳秒部分
- **frame_id: str** - 坐标系标识符, 说明点云坐标的参考坐标系
- **points: Point3DVector** - 相机捕获的三维点云数据列表, 每个点包含 X、Y、Z 坐标

4.7 机械臂相关数据

● 枚举类型

4.7.1 轴参数索引 (AxisParameterIndex)

```
AxisParameterIndex.KPositiveLimitation # 正向极限  
AxisParameterIndex.KNegativeLimitation # 负向极限
```

4.7.2 手臂绑定模式 (ArmBindMode)

```
ArmBindMode.ArmBindWithNull # 无绑模式  
ArmBindMode.LeftArmBindWithWaist # 左臂与腰绑定  
ArmBindMode.RightArmBindWithWaist # 右臂与腰绑定  
ArmBindMode.DoubleArmBindWithWaist # 双臂与腰绑定
```

● 数据结构

4.7.3 负载信息 (LoadInfo)

```
class LoadInfo:
    loadId: int          # 负载 ID
    loadType: int        # 负载类型
    loadPosition: FloatVector # 负载位置坐标
    weight: float        # 负载重量
    loadDimensions: LoadInfo.LoadDimensions # 负载尺寸信息

    class LoadDimensions:
        length: float # 长度
        width: float  # 宽度
        height: float # 高度
```

4.7.4 机械臂世界坐标数据 (RobotWorldsData)

```
class RobotWorldsData:
    robotId: int          # 机械臂 ID
    worldPos: FloatVector # XYZUVW 坐标
    needCallback: bool    # 是否需要回调
    accTime: float        # 临时空间运动加速时间, 0 使用默认值
    decTime: float        # 临时空间运动减速时间, 0 使用默认值
    maxLineSpeed: float   # 最大线速度 (m/s), 0 使用默认值
    specifyGlobalSpeed: float # 临时全局速度比值 (0~1), 0 使用当前全局速度
```

4.7.5 轴运动结束回调数据 (MoveJointsEndData)

```
class MoveJointsEndData:
    robotId: int          # 机械臂 ID
    end: bool            # 轴运动是否结束 (True = 运动结束, False = 运动中)
```

4.7.6 世界坐标运动结束回调数据 (MoveWorldsEndData)

```
class MoveWorldsEndData:
    robotId: int          # 机械臂 ID
    end: bool             # 世界坐标运动是否结束 (True = 运动结束, False = 运动中)
```

4.8 Vector 类型说明

YouiPilot SDK 提供了专用的 Vector 类型用于在 Python 和 C++ 之间传递数组数据。

Vector 类型使用注意事项:

- Vector 类型专门用于与 C++ 接口交互，提高数据传输效率
- 支持 Python 列表的基本操作：append、extend、索引访问、遍历
- 在传递给 API 时，必须使用对应的 Vector 类型，不能直接使用 Python list
- 可以通过 list() 函数转换为 Python 列表：python_list = list(vector)

4.8.1 浮点数向量 (FloatVector)

```
# 创建和使用 FloatVector
floats = pypilot.FloatVector()
floats.append(1.5)
floats.append(2.7)
floats.extend([3.1, 4.2])

# 访问元素
first_value = floats[0] # 1.5
length = len(floats)    # 4

# 遍历
for value in floats:
```

```
print(f"浮点数值: {value}")
```

4.8.2 双精度浮点数向量 (DoubleVector)

```
# 创建和使用 DoubleVector
doubles = pypilot.DoubleVector()
doubles.append(1.23456789)
doubles.append(2.34567891)

# 用于精确的浮点数计算
for value in doubles:
    print(f"双精度值: {value:.10f}")
```

4.8.3 整数向量 (IntVector)

```
# 创建和使用 IntVector
ints = pypilot.IntVector()
ints.append(10)
ints.append(20)
ints.extend([30, 40, 50])

# 用于存储数字 ID、状态码等
for i, value in enumerate(ints):
    print(f"索引{i}: {value}")
```

4.8.4 字符串向量 (StringVector)

```
# 创建和使用 StringVector
strings = pypilot.StringVector()
strings.append("节点 1")
strings.append("节点 2")
strings.extend(["节点 3", "节点 4"])
```

```
# 用于存储 ID 列表、名称列表等
for string in strings:
    print(f"字符串: {string}")
```

4.8.5 布尔向量 (BoolVector)

```
# 创建和使用 BoolVector
bools = pypilot.BoolVector()
bools.append(True)
bools.append(False)
bools.extend([True, True, False])

# 用于存储状态标志、传感器状态等
for i, value in enumerate(bools):
    print(f"传感器{i}: {'开启' if value else '关闭'}")
```

4.8.6 字符串映射 (StringMap)

```
# 创建和使用 StringMap
string_map = pypilot.StringMap()
string_map["map_001"] = "a1b2c3d4e5f6"
string_map["config"] = "enabled"

# 访问元素
md5_value = string_map["map_001"]
print(f"MD5 值: {md5_value}")

# 检查键是否存在
if "config" in string_map:
    print(f"配置值: {string_map['config']}")

# 遍历所有键值对
for key in string_map:
    print(f"键: {key}, 值: {string_map[key]}")
```

第5章 错误处理

5.1 错误码说明

大部分 API 接口都返回结果码，通常遵循以下规则：

- 0: 成功
- 非 0: 失败
 - -1: 接口调用失败
 - 1: 数据格式无效
 - 2: 任务/动作/数据操作无效
 - 3: 操作参数错误
 - 4: 文件未找到
 - 5: 数据操作权限不足
 - 6: 数据操作失败
 - 99: 服务器内部错误

元组类型的返回值第一个元素为结果码，后续元素为业务数据。

5.2 常见错误处理模式

```
def handle_api_result(result_code, success_msg, error_msg):  
    if result_code == 0:  
        print(success_msg)  
        return True  
    else:  
        print(f"{error_msg}, 错误码: {result_code}")
```

```
return False
```

```
# 使用示例
# 单返回值接口
result = sdk.armOpenCom("COM1", 9600)
handle_api_result(result, "串口打开成功", "串口打开失败")

# 元组返回接口
result_code, joints = sdk.armGetJoints(0)
if handle_api_result(result_code, "关节获取成功", "关节获取失败"):
    print(f"当前关节: {joints}")
```

5.3 异常处理

```
try:
    sdk = pypilot.PilotSDK("192.168.1.100")
    if not sdk.start():
        raise Exception("SDK 启动失败")

    # 执行各种操作...

except Exception as e:
    print(f"发生异常: {e}")
    # 执行清理操作
    if 'sdk' in locals():
        sdk.stop()
```

5.4 连接状态检查

```
def check_connection(sdk):
    """检查连接状态"""
    try:
        status = sdk.getConnectionStatus()
        # 解析 JSON 状态
        import json
```

```
        status_data = json.loads(status)
        return status_data.get('connected', False)
    except:
        return False

# 使用示例
if check_connection(sdk):
    print("连接正常")
else:
    print("连接异常，需要重新连接")
```

第6章 联系我们

如使用 SDK 过程中遇到了该文档未记载的问题，请联系 YouiPilot SDK 技术支持团队。

企业微信



钉钉

